

Práctica 3: OpenSSL y Criptografía Pública

Javier López Cano

Mario García Pascual

December 16, 2020

1 OpenSSL

OpenSSL es una librería disponible para múltiples lenguajes de programación que proporciona la implementación de un enorme número de criptosistemas y métodos de cifrado tanto simétricos como asimétricos, así como algunas herramientas auxiliares para establecer los parámetros y protocolos de la comunicación entre el emisor y receptor.

Además, OpenSSL también cuenta con una herramienta para ejecutar estos cifrados y procesos desde comandos en el terminal de Linux, y es en esto en lo que nos vamos a centrar.

Explorando los distintos comandos de OpenSSL, vemos que OpenSSL muestra un listado de los criptosistemas completos disponibles mediante el comando "ciphers". Estos criptosistemas son completos, en el sentido que proporcionan todo lo necesario para la comunicación, incluyendo el protocolo de envío, algoritmo de cifrado asimétrico para la generación de claves y autenticación de las partes, algoritmo simétrico para el cifrado del mensaje a enviar, e incluso un algoritmo de firma MAC para verificar la integridad del mensaje enviado. Sin embargo, los criptosistemas que ofrece OpenSSL mediante este comando no son siempre los mismos, sino que dependen tanto de la versión de OpenSSL como del sistema en que se esté ejecutando. Ejecutamos ahora el comando:

```
1 openssl ciphers -v
```

Y vemos que, en nuestro caso OpenSSL tiene disponibles los siguientes cifradores completos:

```

javi@javi-TP301UA:~/Documentos/Cripto/p3$ openssl ciphers -v
TLS_AES_256_GCM_SHA384 TLSv1.3 Kx=any Au=any Enc=AESGCM(256) Mac=AEAD
TLS_CHACHA20_POLY1305_SHA256 TLSv1.3 Kx=any Au=any Enc=CHACHA20/POLY1305(256) Mac=AEAD
TLS_AES_128_GCM_SHA256 TLSv1.3 Kx=any Au=any Enc=AESGCM(128) Mac=AEAD
ECDHE-ECDSA-AES256-GCM-SHA384 TLSv1.2 Kx=ECDH Au=ECDSA Enc=AESGCM(256) Mac=AEAD
ECDHE-RSA-AES256-GCM-SHA384 TLSv1.2 Kx=ECDH Au=RSA Enc=AESGCM(256) Mac=AEAD
DHE-RSA-AES256-GCM-SHA384 TLSv1.2 Kx=DH Au=RSA Enc=AESGCM(256) Mac=AEAD
ECDHE-ECDSA-CHACHA20-POLY1305 TLSv1.2 Kx=ECDH Au=ECDSA Enc=CHACHA20/POLY1305(256) Mac=AEAD
ECDHE-RSA-CHACHA20-POLY1305 TLSv1.2 Kx=ECDH Au=RSA Enc=CHACHA20/POLY1305(256) Mac=AEAD
DHE-RSA-CHACHA20-POLY1305 TLSv1.2 Kx=DH Au=RSA Enc=CHACHA20/POLY1305(256) Mac=AEAD
ECDHE-ECDSA-AES128-GCM-SHA256 TLSv1.2 Kx=ECDH Au=ECDSA Enc=AESGCM(128) Mac=AEAD
ECDHE-RSA-AES128-GCM-SHA256 TLSv1.2 Kx=ECDH Au=RSA Enc=AESGCM(128) Mac=AEAD
DHE-RSA-AES128-GCM-SHA256 TLSv1.2 Kx=DH Au=RSA Enc=AESGCM(128) Mac=AEAD
ECDHE-ECDSA-AES256-SHA384 TLSv1.2 Kx=ECDH Au=ECDSA Enc=AES(256) Mac=SHA384
ECDHE-RSA-AES256-SHA384 TLSv1.2 Kx=ECDH Au=RSA Enc=AES(256) Mac=SHA384
DHE-RSA-AES256-SHA256 TLSv1.2 Kx=DH Au=RSA Enc=AES(256) Mac=SHA256
ECDHE-ECDSA-AES128-SHA256 TLSv1.2 Kx=ECDH Au=ECDSA Enc=AES(128) Mac=SHA256
ECDHE-RSA-AES128-SHA256 TLSv1.2 Kx=ECDH Au=RSA Enc=AES(128) Mac=SHA256
DHE-RSA-AES128-SHA256 TLSv1.2 Kx=DH Au=RSA Enc=AES(128) Mac=SHA256
ECDHE-ECDSA-AES256-SHA TLSv1 Kx=ECDH Au=ECDSA Enc=AES(256) Mac=SHA1
ECDHE-RSA-AES256-SHA TLSv1 Kx=ECDH Au=RSA Enc=AES(256) Mac=SHA1
DHE-RSA-AES256-SHA SSLv3 Kx=DH Au=RSA Enc=AES(256) Mac=SHA1
ECDHE-ECDSA-AES128-SHA TLSv1 Kx=ECDH Au=ECDSA Enc=AES(128) Mac=SHA1
ECDHE-RSA-AES128-SHA TLSv1 Kx=ECDH Au=RSA Enc=AES(128) Mac=SHA1
DHE-RSA-AES128-SHA SSLv3 Kx=DH Au=RSA Enc=AES(128) Mac=SHA1
RSA-PSK-AES256-GCM-SHA384 TLSv1.2 Kx=RSAPSK Au=RSA Enc=AESGCM(256) Mac=AEAD
DHE-PSK-AES256-GCM-SHA384 TLSv1.2 Kx=DHEPSK Au=PSK Enc=AESGCM(256) Mac=AEAD
RSA-PSK-CHACHA20-POLY1305 TLSv1.2 Kx=RSAPSK Au=RSA Enc=CHACHA20/POLY1305(256) Mac=AEAD
DHE-PSK-CHACHA20-POLY1305 TLSv1.2 Kx=DHEPSK Au=PSK Enc=CHACHA20/POLY1305(256) Mac=AEAD
ECDHE-PSK-CHACHA20-POLY1305 TLSv1.2 Kx=ECDEPSK Au=PSK Enc=CHACHA20/POLY1305(256) Mac=AEAD
AES256-GCM-SHA384 TLSv1.2 Kx=RSA Au=RSA Enc=AESGCM(256) Mac=AEAD
PSK-AES256-GCM-SHA384 TLSv1.2 Kx=PSK Au=PSK Enc=AESGCM(256) Mac=AEAD
PSK-CHACHA20-POLY1305 TLSv1.2 Kx=PSK Au=PSK Enc=CHACHA20/POLY1305(256) Mac=AEAD
RSA-PSK-AES128-GCM-SHA256 TLSv1.2 Kx=RSAPSK Au=RSA Enc=AESGCM(128) Mac=AEAD
DHE-PSK-AES128-GCM-SHA256 TLSv1.2 Kx=DHEPSK Au=PSK Enc=AESGCM(128) Mac=AEAD
AES128-GCM-SHA256 TLSv1.2 Kx=RSA Au=RSA Enc=AESGCM(128) Mac=AEAD
PSK-AES128-GCM-SHA256 TLSv1.2 Kx=PSK Au=PSK Enc=AESGCM(128) Mac=AEAD
AES256-SHA256 TLSv1.2 Kx=RSA Au=RSA Enc=AES(256) Mac=SHA256
AES128-SHA256 TLSv1.2 Kx=RSA Au=RSA Enc=AES(128) Mac=SHA256
ECDHE-PSK-AES256-CBC-SHA384 TLSv1 Kx=ECDEPSK Au=PSK Enc=AES(256) Mac=SHA384
ECDHE-PSK-AES256-CBC-SHA TLSv1 Kx=ECDEPSK Au=PSK Enc=AES(256) Mac=SHA1
SRP-RSA-AES-256-CBC-SHA SSLv3 Kx=SRP Au=RSA Enc=AES(256) Mac=SHA1
SRP-AES-256-CBC-SHA SSLv3 Kx=SRP Au=SRP Enc=AES(256) Mac=SHA1
RSA-PSK-AES256-CBC-SHA384 TLSv1 Kx=RSAPSK Au=RSA Enc=AES(256) Mac=SHA384
DHE-PSK-AES256-CBC-SHA384 TLSv1 Kx=DHEPSK Au=PSK Enc=AES(256) Mac=SHA384
RSA-PSK-AES256-CBC-SHA SSLv3 Kx=RSAPSK Au=RSA Enc=AES(256) Mac=SHA1
DHE-PSK-AES256-CBC-SHA SSLv3 Kx=DHEPSK Au=PSK Enc=AES(256) Mac=SHA1
AES256-SHA SSLv3 Kx=RSA Au=RSA Enc=AES(256) Mac=SHA1
PSK-AES256-CBC-SHA384 TLSv1 Kx=PSK Au=PSK Enc=AES(256) Mac=SHA384
PSK-AES256-CBC-SHA SSLv3 Kx=PSK Au=PSK Enc=AES(256) Mac=SHA1
ECDHE-PSK-AES128-CBC-SHA256 TLSv1 Kx=ECDEPSK Au=PSK Enc=AES(128) Mac=SHA256
ECDHE-PSK-AES128-CBC-SHA TLSv1 Kx=ECDEPSK Au=PSK Enc=AES(128) Mac=SHA1
SRP-RSA-AES-128-CBC-SHA SSLv3 Kx=SRP Au=RSA Enc=AES(128) Mac=SHA1
SRP-AES-128-CBC-SHA SSLv3 Kx=SRP Au=SRP Enc=AES(128) Mac=SHA1
RSA-PSK-AES128-CBC-SHA256 TLSv1 Kx=RSAPSK Au=RSA Enc=AES(128) Mac=SHA256
DHE-PSK-AES128-CBC-SHA256 TLSv1 Kx=DHEPSK Au=PSK Enc=AES(128) Mac=SHA256
RSA-PSK-AES128-CBC-SHA SSLv3 Kx=RSAPSK Au=RSA Enc=AES(128) Mac=SHA1
DHE-PSK-AES128-CBC-SHA SSLv3 Kx=DHEPSK Au=PSK Enc=AES(128) Mac=SHA1
AES128-SHA SSLv3 Kx=RSA Au=RSA Enc=AES(128) Mac=SHA1
PSK-AES128-CBC-SHA256 TLSv1 Kx=PSK Au=PSK Enc=AES(128) Mac=SHA256
PSK-AES128-CBC-SHA SSLv3 Kx=PSK Au=PSK Enc=AES(128) Mac=SHA1

```

Estos son muchos cifrados, pero es relativamente fácil explicarlos si nos fijamos en cada uno de los datos que nos muestra de cada uno de ellos. En primer lugar nos indica el "nombre" con que OPenSSL identifica el cifrado, tras esto nos indica la versión de protocolo de envío SSL o TLS con la que es compatible. Por último nos indica los 4 parámetros más interesantes que nos indican el funcionamiento del cifrado:

- Kx indica el protocolo empleado para el intercambio de la clave simétrica entre el emisor y el receptor.

- Au indica el protocolo empleado para la autenticación (firma digital) en el envío del mensaje (no de la clave, pues este es Kx).
- Enc indica el algoritmo de cifrado del propio mensaje.
- Mac indica el tipo de firma (Hashing) empleada para verificar la integridad del fichero.

Vamos a explicar brevemente cada uno de los posibles valores que toman estas variables, para poder así entender el funcionamiento de cada uno de los cifrados que nos proporciona la librería.

La variable Kx puede tomar los valores any, ECDH, DH, RSAPSK, DHEPSK, ECDHEPSK, RSA, PSK y SRP. Estos indican:

- any: indica que el cifrador no proporciona ningún algoritmo para el traspaso de clave.
- DH (Diffie-Hellman): en este caso las claves simétricas se envían por un canal inseguro.
- ECDH (Elliptic Curve-DH): este protocolo envía las claves por un canal inseguro al igual que DH, pero con la particularidad de que para que la comunicación por este canal inseguro sea secreta, se envía "encriptada" mediante un par de claves pública-privada basada en curvas elípticas.
- DHE (DH-Efimeral): Igual que el DH, pero con claves temporales en lugar de permanentes.
- ECDHE (ECDH-Efimeral): Igual que el ECDH, pero con claves temporales en lugar de permanentes.
- PSK (Pre-Shared Keys): Las claves simétricas deben haberse compartido previamente entre emisor y receptor.
- RSA: El intercambio de claves se realiza encriptado mediante un par de claves pública-privada generados por RSA.
- ECDHEPSK: Emplea ECDHE pero la clave enviada por el canal inseguro requiere una autenticación con una clave precompartida entre emisor y receptor.
- DHEPSK: DHE con autenticación mediante clave precompartida.
- RSAPSK: Se emplean claves RSA pero la clave pública inicial mandada por el servidor al cliente se envía cifrada con una clave precompartida.
- SRP (Secure Remote Password): este protocolo permite generar una clave simétrica para una única sesión sin necesidad de almacenar una contraseña del usuario.

La variable Au puede tomar los valores any, ECDSA, RSA, PSK, SRP. Estos valores indican los mismos protocolos que en el caso de Kx, excepto ECDSA.

- ECDSA (Elliptic Curve Digital Signature Algorithm): Se basa en el uso de una firma digital para la autenticación de las partes, con la peculiaridad de que esta firma se calcula empleando un par de claves pública-privada generadas mediante curvas elípticas.

La variable Enc Puede tomar los valores AES, AESGCM y CHACHA20/POLY1305 seguidos de un número entre paréntesis (256 o 128) que indica el tamaño del bloque.

- AES: Algoritmo AES estudiado en clase.
- AESGCM: Algoritmo AES en modo GCM (Galois Counter Mode).
- CHACHA20/POLY1305: Se trata del algoritmo de cifrado de flujo CHACHA20 con una firma POLY1305. El CHACHA20 es un cifrado de flujo, cuya generación de subclaves se basa en un generador pseudoaleatorio que emplea el XOR, la rotación de bits y sumas de 32 bits. La firma POLY1305 es una firma para comprobación de integridad del fichero basada en el algoritmo AES, proporcionando una seguridad muy similar a este.

En el caso de la variable "Enc" nos ha llamado mucho la atención que no se nos haya mostrado en nuestro caso ningún criptosistema que emplee cifrados simétricos extremadamente comunes y populares como DES, IDEA o Camellia.

Por último la variable Mac toma los valores AEAD y SHA(seguido de un número):

- AEAD (Authenticated Encryption with Additional Data): permite comprobar la integridad del mensaje tanto tras descifrarlo como antes de descifrarlo cuando aún sigue cifrado. Para ello añade datos extra al texto cifrado así como al contexto en que va a aparecer, para imposibilitar el "copy-paste" del mensaje.
- SHA (Secure Hash Algorithm): es una familia de algoritmos paraa firma del mensaje mediante una función Hash, donde el número que sigue a "SHA" indica el algoritmo concreto empleado, donde SHA0 es el más antiguo, SHA1 el siguiente, y SHA3 el más moderno, siendo todos los demás números funciones o variantes del SHA2.

De este modo, el criptosistema:

$$ECDHE - ECDSA - AES128 - SHA - TLSv1 \quad Kx = ECDH$$

$$Au = ECDSA \quad Enc = AES(128) \quad Mac = SHA1$$

Es un cifrado AES con bloques de 128 bits, donde la clave simétrica se comparte mediante un protocolo ECDH, la autenticación de los usuarios al cifrar y descifrar se realiza mediante ECDSA, y la firma "MAC" para comprobar la integridad del fichero se realiza mediante una función hash generada por SHA-1.

Procedemos ahora a ver no los criptosistemas completos que nos ofrece OpenSSL, sino los cifrados concretos, tanto simétricos como asimétricos que nos permite ejecutar directamente sin necesidad de escoger un criptosistema de los proporcionados. Para ello observamos todos los comandos que nos proporciona OpenSSL ejecutando el comando de ayuda:

```
1 openssl help
```

Y obtenemos la siguiente salida:

```

eps@labvirteps:~$ openssl help
openssl:Error: 'help' is an invalid command.

Standard commands
asn1parse      ca              ciphers         cms
crl            crt2pkcs7      dgst            dh
dhparam        dsa            dsaparam        ec
ecparam        enc            engine          errstr
gendh          gendsa         genpkey         genrsa
nseq           ocsf           passwd          pkcs12
pkcs7          pkcs8          pkey            pkeyparam
pkeyutl        prime          rand            req
rsa            rsautl         s_client        s_server
s_time         sess_id        smime           speed
spkac          srp            ts              verify
version        x509

Message Digest commands (see the 'dgst' command for more details)
md4            md5            mdc2            rmd160
sha            sha1

Cipher commands (see the 'enc' command for more details)
aes-128-cbc    aes-128-ecb    aes-192-cbc    aes-192-ecb
aes-256-cbc    aes-256-ecb    base64         bf
bf-cbc        bf-cfb        bf-ecb         bf-ofb
camellia-128-cbc camellia-128-ecb camellia-192-cbc camellia-192-ecb
camellia-256-cbc camellia-256-ecb cast            cast-cbc
cast5-cbc      cast5-cfb      cast5-ecb      cast5-ofb
des            des-cbc        des-cfb        des-ecb
des-ede        des-ede-cbc    des-ede-cfb    des-ede-ofb
des-ede3       des-ede3-cbc   des-ede3-cfb   des-ede3-ofb
des-ofb        des3           desx           idea
idea-cbc       idea-cfb       idea-ecb       idea-ofb
rc2            rc2-40-cbc     rc2-64-cbc     rc2-cbc
rc2-cfb        rc2-ecb        rc2-ofb        rc4
rc4-40         seed           seed-cbc       seed-cfb
seed-ecb       seed-ofb

```

Al observar esta salida vemos que los cifrados simétricos se encuentran bajo el comando "enc", mientras que los asimétricos se encuentran directamente como comandos de OpenSSL como "rsa".

1.1 Cifrados Simétricos

Para obtener el listado completo de todos los cifrados simétricos que ofrece OpenSSL, accedemos a la ayuda del comando "enc", ejecutando:

```
1 openssl enc -h
```

De este modo se nos imprime el siguiente listado:

Cipher Types		
-aes-128-cbc	-aes-128-ccm	-aes-128-cfb
-aes-128-cfb1	-aes-128-cfb8	-aes-128-ctr
-aes-128-ecb	-aes-128-ofb	-aes-192-cbc
-aes-192-ccm	-aes-192-cfb	-aes-192-cfb1
-aes-192-cfb8	-aes-192-ctr	-aes-192-ecb
-aes-192-ofb	-aes-256-cbc	-aes-256-ccm
-aes-256-cfb	-aes-256-cfb1	-aes-256-cfb8
-aes-256-ctr	-aes-256-ecb	-aes-256-ofb
-aes128	-aes192	-aes256
-bf	-bf-cbc	-bf-cfb
-bf-ecb	-bf-ofb	-blowfish
-camellia-128-cbc	-camellia-128-cfb	-camellia-128-cfb1
-camellia-128-cfb8	-camellia-128-ecb	-camellia-128-ofb
-camellia-192-cbc	-camellia-192-cfb	-camellia-192-cfb1
-camellia-192-cfb8	-camellia-192-ecb	-camellia-192-ofb
-camellia-256-cbc	-camellia-256-cfb	-camellia-256-cfb1
-camellia-256-cfb8	-camellia-256-ecb	-camellia-256-ofb
-camellia128	-camellia192	-camellia256
-cast	-cast-cbc	-cast5-cbc
-cast5-cfb	-cast5-ecb	-cast5-ofb
-des	-des-cbc	-des-cfb
-des-cfb1	-des-cfb8	-des-ecb
-des-ede	-des-ede-cbc	-des-ede-cfb
-des-ede-ofb	-des-ede3	-des-ede3-cbc
-des-ede3-cfb	-des-ede3-cfb1	-des-ede3-cfb8
-des-ede3-ofb	-des-ofb	-des3
-desx	-desx-cbc	-id-aes128-CCM
-id-aes128-wrap	-id-aes192-CCM	-id-aes192-wrap
-id-aes256-CCM	-id-aes256-wrap	-id-smime-alg-CMS3DESwrap
-idea	-idea-cbc	-idea-cfb
-idea-ecb	-idea-ofb	-rc2
-rc2-40-cbc	-rc2-64-cbc	-rc2-cbc
-rc2-cfb	-rc2-ecb	-rc2-ofb
-rc4	-rc4-40	-seed
-seed-cbc	-seed-cfb	-seed-ecb
-seed-ofb		

Vemos, por tanto, que dispone de diversos algoritmos de cifrado simétrico, con varios modos y tamaños de clave para cada uno de ellos. En general, los algoritmos disponibles son:

- AES y DES: algoritmos vistos en clase y en prácticas anteriores de forma detallada. Se encuentran disponibles en prácticamente todos sus distintos modos, y con diversos tamaños de clave entre 128 y 256 bits.
- DESX: es una variante del DES que implementa un blanqueo de claves para aumentar la seguridad frente a ataques por fuerza bruta. Básicamente consiste en aplicar un XOR entre una clave y el texto plano antes de comenzar a cifrar con DES, tras esto se realiza el cifrado DES con su correspondiente clave, y al finalizar se realiza otro XOR con una tercera clave.
- BF (Blowfish): se trata de un cifrado simétrico de Feistel (al igual que DES) de 16 rondas con bloques de 64 bits, con la peculiaridad de que las SBOXES que emplea no son fijas, sino que dependen de la clave. Este cifrado admite claves de entre 32 y 448 bits. Del mismo modo que otros cifrados de bloque, está disponible en distintos modos, que se caracterizan por distintos modos de proporcionar encadenamiento de bloques, o por el uso de contadores.
- Camellia: es un cifrado simétrico por bloques de 128 bits con claves de 128, 192 y 256 bits. Se basa en un cifrado Feistel de entre 18 y 24 rondas (el número de rondas depende de la longitud de la clave, siendo 18 para claves de 128 bits, y 24 para las otras 2 opciones). Emplea 4 SBOXES no invertibles de

tamaño 8x8 bits, y calculadas mediante transformaciones afines y transformaciones lógicas, por lo que no son invertibles, y se requieren SBOXES tanto de cifrado como de descifrado. además este cifrado tiene la peculiaridad de que cada 6 rondas se realiza una transformación lógica del bloque mediante una función llamada "función FL", que es invertible, para permitir el descifrado. Este cifrado emplea también blanqueamiento de clave, del mismo modo que DESX empleando un XOR antes y otro después del cifrado.

- CAST: es una familia de cifrados Feistel que emplean SBOXES de tamaño muy grande (8x32), divide los bloques a cifrar en sub-bloques de 32, y admite claves de 128, 192 y 256 bits. Los algoritmos de esta familia realizan una combinación de las salidas de 32 bits de sus SBOXES, para generar un efecto avalancha más notable que si se usasen estas salidas directamente sin combinarlas entre sí; además, realizan una rotación a la salida de esta combinación; esta rotación es dependiente de la clave. Existen 2 algoritmos principales dentro de esta familia, que son además los disponibles en OpenSSL, estos son CAST-128 y CAST-256. El primero emplea bloques de 64 bits, y realiza 16 rondas, en concreto este emplea claves de 128 bits; este algoritmo realiza un XOR entre cada sub-bloque y 32 bits de los 37 que conforman la clave de una ronda, y el resultado (de 32 bits) lo divide en 4 bloques de 1 byte, pasando cada uno de ellos por una SBOX y combina las salidas de estas (de 32 bits cada una) de forma no lineal. Tras esto realiza una rotación que depende de los 5 bits restantes de la clave. El segundo algoritmo (CAST-256) emplea bloques de 128 bits y claves de 128, 192 o 256 bits; y realiza 48 rondas en lugar de las 16 del CAST-128.
- IDEA: cifrado por bloques de 64 bits divididos en sub-bloques de 16 con claves de 128 bits. Se trata de un algoritmo que no es de tipo Feistel, por lo que es significativamente distinto a los anteriores. Se basa en el uso de operaciones aritméticas en módulo 2^{16} y operaciones XOR.
- Base64: consiste en dividir el texto plano en bloques de 6 bits y representar en la salida los caracteres ascii correspondientes a cada grupo de 6 bits.
- RC2: cifrado con bloques de 64 bits y clave de tamaño variable. Realiza un cifrado en 18 rondas, basado en Feistel, pero un Feistel no balanceado, es decir, emplea el algoritmo de cifrado Feistel con la peculiaridad de que las 2 mitades de bloque que se van intercambiando en cada ronda no tienen el mismo tamaño.
- RC4: cifrado de flujo cuya generación de subclaves se basa en 2 algoritmos, un primer algoritmo KSA que genera un vector (SBOX) de 256 posiciones a partir de la clave inicial, y un segundo algoritmo PRGA, que, a partir de este vector, genera las subclaves de cifrado, modificando ligeramente el vector en el proceso de modo que la siguiente llamada a este no de como resultado la misma clave, es este segundo algoritmo PRGA, el que funciona en efecto como generador de claves.
- SEED: cifrado de tipo Feistel con bloques de 128 bits y claves de la misma longitud. realiza 18 rondas de cifrado con 2 SBOXES de tamaño 8x8 bits generadas mediante operaciones de exponenciación modular. Este algoritmo tiene la peculiaridad de que su función F, es a su vez otro algoritmo de tipo Feistel, que opera sobre los 64 bits de la mitad de bloque correspondiente a la ronda, dividiéndola en 2 mitades de 32 bits cada una, y aplicando sobre ellas un algoritmo de Feistel, donde la función F ya no es un algoritmo de feistel, sino una simple combinación de una sustitución con varias SBOXES cuyas salidas se combinan de forma no lineal.

Vamos a ejecutar un cifrado y descifrado con algunos de ellos. Comenzamos con el DES, empleando el modo ECB, y probaremos una clave débil del DES, por ejemplo, todo 0s. Para ello ciframos El Quijote (fichero "q") con el siguiente comando:

```
1 openssl enc -des-ecb -K 0000000000000000 -in q -out q_cif_weak_des
```

Con esto obtenemos un fichero "q_cif_weak_des" cifrado no legible por lectores de texto pues no hemos seleccionado la opción "base64" que imprimiría caracteres ASCII. Para comprobar que se trata de una clave débil, probamos a cifrar este fichero de nuevo con la clave 0:

```
1 openssl enc -des-ecb -K 0000000000000000 -in q_cif_weak_des -out q_cif_weak_des2
```

Y obtenemos el fichero "q_cif_weak_des2" que, efectivamente, es de nuevo el archivo original sin cifrar.

Vamos a probar ahora alguno de los pares de claves semidébiles, por ejemplo, el par (0x011F011F010E010E y 0x1F011F010E010E01)

```
1 openssl enc -des-ecb -K 011f011f010e010e -in q -out q_cif_semi_des
2 openssl enc -des-ecb -K 1f011f010e010e01 -in q_cif_semi_des -out q_cif_semi_des2
```

Y de nuevo vemos que el fichero "q_cif_semi_des2" es igual al archivo original "q".

Probamos ahora el cifrado y descifrado con blowfish, IDEA y RC4, usando la clave 1234123412341234 y el iv abcdabcdabcdabcd:

```
1 openssl enc -blowfish -iv abcdabcdabcdabcd -K 1234123412341234 -in q -out q_cif_BF
2 openssl enc -d -blowfish -iv abcdabcdabcdabcd -K 1234123412341234 -in q_cif_BF -out q_dec_BF
```

```
1 openssl enc -idea -iv abcdabcdabcdabcd -K 1234123412341234 -in q -out q_cif_idea
2 openssl enc -d -idea -iv abcdabcdabcdabcd -K 1234123412341234 -in q_cif_idea -out q_dec_idea
```

```
1 openssl enc -RC4 -K 1234123412341234 -in q -out q_cif_RC4
2 openssl enc -d -RC4 -K 1234123412341234 -in q_cif_RC4 -out q_dec_RC4
```

Y vemos que todos ellos cifran y descifran correctamente (todos los ficheros cifrados y descifrados se proporcionan en los archivos de la práctica).

1.2 Cifrados Asimétricos

Observando la imagen del comando de ayuda de OpenSSL mostrada anteriormente:

```

eps@labvirteps:~$ openssl help
openssl:Error: 'help' is an invalid command.

Standard commands
asn1parse      ca              ciphers         cms
crl            crl2pkcs7      dgst            dh
dhparam        dsa            dsaparam        ec
ecparam        enc            engine          errstr
gendh          gendsa         genpkey         genrsa
nseq           ocsf           passwd          pkcs12
pkcs7          pkcs8          pkey            pkeyparam
pkeyutl        prime          rand            req
rsa            rsautl         s_client        s_server
s_time         sess_id        smime           speed
spkac          srp            ts              verify
version        x509

Message Digest commands (see the `dgst' command for more details)
md4            md5            mdc2            rmd160
sha            sha1

Cipher commands (see the `enc' command for more details)
aes-128-cbc    aes-128-ecb    aes-192-cbc    aes-192-ecb
aes-256-cbc    aes-256-ecb    base64         bf
bf-cbc        bf-cfb        bf-ecb         bf-ofb
camellia-128-cbc camellia-128-ecb camellia-192-cbc camellia-192-ecb
camellia-256-cbc camellia-256-ecb cast           cast-cbc
cast5-cbc     cast5-cfb     cast5-ecb     cast5-ofb
des           des-cbc      des-cfb      des-ecb
des-ede      des-ede-cbc  des-ede-cfb  des-ede-ofb
des-ede3     des-ede3-cbc des-ede3-cfb  des-ede3-ofb
des-ofb      des3         desx         idea
idea-cbc     idea-cfb     idea-ecb     idea-ofb
rc2          rc2-40-cbc   rc2-64-cbc   rc2-cbc
rc2-cfb      rc2-ecb     rc2-ofb      rc4
rc4-40       seed        seed-cbc     seed-cfb
seed-ecb     seed-ofb

```

Podemos observar en los comandos que los algoritmos de cifrado asimétrico que proporciona esta herramienta son:

- RSA: algoritmo de cifrado asimétrico basado en el uso de un par de claves público-privada (de cifrado y descifrado respectivamente) generadas a partir de las propiedades del producto de números primos. Para generar estas claves se escogen 2 números primos a y b y se calcula su producto m . Mediante la función de Euler calculamos $\phi(m)$ y buscamos un número "pub" menor que $\phi(m)$ y coprimo con este, que será el exponente de la clave pública. Hallamos ahora "priv" que será el exponente de la clave privada, y este es el inverso multiplicativo de "pub" módulo $\phi(m)$. Una vez tenemos estos valores, la clave pública (de cifrado) será (m, pub) y la privada (de descifrado) será (m, priv) , estas claves cumplen que:

$$x = x^{\text{priv} \cdot \text{pub}} \pmod{m} \text{ para todo } x \text{ en } \mathbb{Z}_m$$

Entonces ciframos el texto con la clave pública del receptor $c = x^{\text{pub}} \pmod{m}$ y este lo podrá descifrar con su clave privada $x = c^{\text{priv}} \pmod{m}$.

- DSA: se trata de nuevo de un algoritmo de autenticación basado en un par de claves pública-privada, pero no es un algoritmo de cifrado, pues no modifica el mensaje sino que tan solo añade una firma.
- DH: protocolo explicado anteriormente en la introducción de OpenSSL, que permite a los usuarios de la comunicación establecer una clave común de forma segura, a través de un canal inseguro. Se establecen 2 números públicos, "a" que es un primo aleatorio, y "f" que es un número menor que "a" y coprimo con este. Una de las partes genera un entero aleatorio "x" y calcula su "equivalente público" como $X = f^x \text{ mod } a$, y envía este al otro usuario de la comunicación. Este segundo usuario hace lo mismo con otra clave "Y". De este modo, ambos pueden obtener una clave común $k = f^{xy} \text{ (mod } a) = X^y \text{ (mod } a) = Y^x \text{ (mod } a)$. Mientras que posibles atacantes, pese a conocer "X", "Y", "p" y "f", al no poder inferir de ellos "x" e "y", no pueden hallar la clave. De este modo se ha logrado compartir la clave de forma segura.
- EC: algoritmo basado de nuevo en un par de claves pública-privada, en este caso, generadas a partir de curvas elípticas. Para ello, se escoge un punto P de una curva elíptica y se hace público. Uno de los usuarios de la comunicación calcula su clave privada "x" de forma aleatoria, y su clave pública $X = k \cdot P$ y esta la hace pública. El otro usuario de la comunicación realiza el mismo proceso con sus claves "y" e "Y". De este modo, ambos pueden inferir una clave común $K = x \cdot Y = y \cdot X = x \cdot y \cdot P$. La fortaleza de este algoritmo radica en la alta complejidad de los cálculos con puntos de curvas elípticas que hace muy complicado y costoso hallar la clave privada "x" a partir de los valores de "X" y "P".

Vamos a probar estos algoritmos de nuevo cifrando El Quijote (fichero "q"), vamos a realizar esta prueba con DH y EC, ya que DSA no sirve para cifrar, sino que solo firma, y RSA lo realizaremos en el siguiente apartado.

Comenzamos con Diffie Hellman:

1. En primer lugar generamos los parámetros del algoritmo DH, para ello OpenSSL nos proporciona un generador de parámetros, por lo que no debemos escogerlos manualmente, sino ejecutar el siguiente comando:

```
1 openssl genpkey -genparam -algorithm DH -out params
```

Este comando genera los parámetros y los almacena en el fichero "params", que tiene el siguiente aspecto:

```
|-----BEGIN DH PARAMETERS-----
MIGHAoGBALU+Y2YqZ6eeTCUY2/ZBisjFh6Yr89YDFlIck0oZv1N22un5To7tUkOX
W65IkjJP68F84FSIeP4iHLxWXcTt+uTc4RvmDuh93nlBbp/onF19FlmfEEc8Lkyy
yZcb/v5ykF7xhCfVRfVTvj4m85PGxs50ZHfJ4JAZ07sSx5jiIB6LAgEC
-----END DH PARAMETERS-----
```

2. Ambos usuarios deben usar este fichero de parametros para generar sus claves pública y privada, para ello realizan los siguientes comandos: Usuario1:

```
1 openssl genpkey -paramfile params -out clave1
```

Usuario2:

```
1 openssl genpkey -paramfile params -out clave2
```

Ahora "clave1" tendrá las claves pública y privada del usuario1, y "clave2" las del usuario2. Para poder ver estos ficheros debemos ejecutar el comando:

```
1 openssl pkey -in clave1 -text -noout
```

Con esto vemos que el fichero "clave1" tiene el siguiente aspecto:

```

eps@labvirtips:~/Escritorio/Cripto/p3/pruebasDH$ openssl pkey -in clave1 -text -noout
DH Private-Key: (1024 bit)
private-key:
 41:10:61:e7:3c:87:ab:9a:78:7f:25:e4:86:3a:31:
 10:ff:a0:60:d9:9b:f7:d1:39:fe:04:67:a6:3b:4b:
 0e:ad:bb:e0:93:ee:60:de:eb:b4:b3:cc:a4:fc:ac:
 cf:00:b7:af:37:bb:a4:b2:b6:dc:5a:23:65:92:a8:
 d3:bf:02:8a:52:20:56:bd:20:9a:d8:66:e7:08:76:
 f2:c2:ad:77:04:a8:f3:db:69:94:7a:fe:fc:86:3a:
 4e:4a:a5:a4:a0:07:1b:19:4b:02:83:59:58:41:e4:
 cc:34:45:44:fd:48:15:1d:af:2b:f6:df:d8:bd:cc:
 c3:fb:5f:55:38:3d:76:e4
public-key:
 5f:7a:ae:cc:bc:f5:be:04:88:b2:7d:80:0b:4b:ef:
 6e:92:07:83:85:88:de:8e:2c:62:16:98:6b:57:ed:
 e9:17:8c:5c:99:1e:62:47:cb:9f:8a:45:50:6f:c7:
 10:18:bc:37:1a:8d:81:45:87:ba:72:e6:a6:e2:ca:
 5e:c3:f8:50:c2:43:25:25:43:3b:86:39:3e:fe:11:
 f7:e5:7a:b7:58:55:18:20:ca:61:97:69:46:13:eb:
 6d:15:19:02:22:0b:97:81:63:bb:53:ba:8b:58:e8:
 18:e5:4d:8c:88:4f:20:f7:3e:bc:4d:58:f3:d8:ea:
 a3:ea:c2:dc:8d:b6:20:6f
prime:
 00:b5:3e:63:66:2a:67:a7:9e:4c:25:18:db:f6:41:
 89:28:c5:87:a6:2b:f3:d6:03:16:52:1c:28:ea:19:
 bf:53:76:da:e9:f9:4e:8e:ed:52:43:97:5b:ae:48:
 92:32:4f:eb:c1:7c:e0:54:88:78:fe:22:1c:bc:56:
 5d:c4:ed:fa:e4:dc:e1:1b:e6:0e:e8:7d:de:79:41:
 6e:9f:e8:9c:5d:7d:16:59:9f:10:47:3c:2e:4c:b2:
 c9:97:1b:fe:fe:72:90:5e:f1:84:27:d5:45:f5:53:
 56:3e:26:f3:93:c6:c6:ce:4e:64:77:c9:e0:90:19:
 d3:bb:12:c7:98:e2:20:1e:8b
generator: 2 (0x2)

```

Y "clave2" tiene un aspecto similar con distintas claves.

3. Cada uno de los usuarios extrae su clave pública a otro fichero para compartirla con el otro, para ello se emplea el comando:

```

1 openssl pkey -in clave1 -pubout -out clave1_pub
2 openssl pkey -in clave2 -pubout -out clave2_pub

```

De este modo las claves públicas quedan en los ficheros "clave1_pub" y "clave2_pub", que los usuarios se intercambian.

4. Cada uno de los usuarios calcula la clave común empleando su clave privada, y la clave pública enviada por el otro usuario, para ello ejecutan:

Usuario1:

```

1 openssl pkeyutl -derive -inkey clave1 -peerkey clave2_pub -out secreto1

```

Usuario2:

```

1 openssl pkeyutl -derive -inkey clave2 -peerkey clave1_pub -out secreto2

```

Tras esto la clave común debe estar guardada en los ficheros "secreto1" y "secreto2", cada uno de los cuales lo tendría uno de los usuarios. Estos ficheros tienen el siguiente aspecto:

```

eps@labvirteps:~/Escritorio/Cripto/p3/pruebasDH$ xxd secreto1
00000000: b10b a5a6 02b3 7f38 c527 90b0 dcac 98a2  ....8.'.....
00000010: 7adf b6de 6570 5d50 9cc8 2c35 b275 725f  z...ep]P...,5.ur_
00000020: ae79 159f 788c 0f4a 2283 3fe2 f120 e77b  .y..x..J".?... .{
00000030: 4ccd 9424 0bae 6624 12c9 a9a1 dc97 274d  L..$..f$.....'M
00000040: afd1 2f4b 9ecb c139 363f 98d3 ff3e 116e  ../K...96?...>.n
00000050: 8f2f 8b86 c8ac d244 0688 6e28 6744 9bdd  ./.....D..n(gD..
00000060: c5b4 8cbd 7fd4 7dd3 6d53 ca46 4186 dadd  ....}.mS.FA...
00000070: 5e56 4fc9 e149 e106 ff02 5f98 73db 4741  ^VO..I...._.s.GA
eps@labvirteps:~/Escritorio/Cripto/p3/pruebasDH$ xxd secreto2
00000000: b10b a5a6 02b3 7f38 c527 90b0 dcac 98a2  ....8.'.....
00000010: 7adf b6de 6570 5d50 9cc8 2c35 b275 725f  z...ep]P...,5.ur_
00000020: ae79 159f 788c 0f4a 2283 3fe2 f120 e77b  .y..x..J".?... .{
00000030: 4ccd 9424 0bae 6624 12c9 a9a1 dc97 274d  L..$..f$.....'M
00000040: afd1 2f4b 9ecb c139 363f 98d3 ff3e 116e  ../K...96?...>.n
00000050: 8f2f 8b86 c8ac d244 0688 6e28 6744 9bdd  ./.....D..n(gD..
00000060: c5b4 8cbd 7fd4 7dd3 6d53 ca46 4186 dadd  ....}.mS.FA...
00000070: 5e56 4fc9 e149 e106 ff02 5f98 73db 4741  ^VO..I...._.s.GA
eps@labvirteps:~/Escritorio/Cripto/p3/pruebasDH$ █

```

Y efectivamente se observa que ambos ficheros tienen el mismo contenido. Ahora se podrían compartir mensajes cifrados con el contenido de ese fichero como clave de forma segura.

Realizamos ahora la prueba con el algoritmo de curvas elípticas, el proceso es muy similar:

1. Generar parámetros de la curva elíptica, para ello escogemos alguna de las curvas disponibles, que podemos ver con el comando:

```
1 openssl ecparam -list_curves
```

```

eps@labvirteps:~/Escritorio/Cripto/p3/pruebasEC$ openssl ecparam -list_curves
secp112r1 : SECG/WTLS curve over a 112 bit prime field
secp112r2 : SECG curve over a 112 bit prime field
secp128r1 : SECG curve over a 128 bit prime field
secp128r2 : SECG curve over a 128 bit prime field
secp160k1 : SECG curve over a 160 bit prime field
secp160r1 : SECG curve over a 160 bit prime field
secp160r2 : SECG/WTLS curve over a 160 bit prime field
secp192k1 : SECG curve over a 192 bit prime field
secp224k1 : SECG curve over a 224 bit prime field
secp224r1 : NIST/SECG curve over a 224 bit prime field
secp256k1 : SECG curve over a 256 bit prime field
secp384r1 : NIST/SECG curve over a 384 bit prime field
secp521r1 : NIST/SECG curve over a 521 bit prime field
prime192v1: NIST/X9.62/SECG curve over a 192 bit prime field
prime192v2: X9.62 curve over a 192 bit prime field
prime192v3: X9.62 curve over a 192 bit prime field
prime239v1: X9.62 curve over a 239 bit prime field
prime239v2: X9.62 curve over a 239 bit prime field
prime239v3: X9.62 curve over a 239 bit prime field
prime256v1: X9.62/SECG curve over a 256 bit prime field
sect113r1 : SECG curve over a 113 bit binary field
sect113r2 : SECG curve over a 113 bit binary field
sect131r1 : SECG/WTLS curve over a 131 bit binary field
sect131r2 : SECG curve over a 131 bit binary field
sect163k1 : NIST/SECG/WTLS curve over a 163 bit binary field
sect163r1 : SECG curve over a 163 bit binary field
sect163r2 : NIST/SECG curve over a 163 bit binary field
sect193r1 : SECG curve over a 193 bit binary field
sect193r2 : SECG curve over a 193 bit binary field
sect233k1 : NIST/SECG/WTLS curve over a 233 bit binary field
sect233r1 : NIST/SECG/WTLS curve over a 233 bit binary field
sect239k1 : SECG curve over a 239 bit binary field
sect283k1 : NIST/SECG curve over a 283 bit binary field
sect283r1 : NIST/SECG curve over a 283 bit binary field
sect409k1 : NIST/SECG curve over a 409 bit binary field
sect409r1 : NIST/SECG curve over a 409 bit binary field
sect571k1 : NIST/SECG curve over a 571 bit binary field
sect571r1 : NIST/SECG curve over a 571 bit binary field
c2pnb163v1: X9.62 curve over a 163 bit binary field
c2pnb163v2: X9.62 curve over a 163 bit binary field
c2pnb163v3: X9.62 curve over a 163 bit binary field
c2pnb176v1: X9.62 curve over a 176 bit binary field
c2tnb191v1: X9.62 curve over a 191 bit binary field
c2tnb191v2: X9.62 curve over a 191 bit binary field
c2tnb191v3: X9.62 curve over a 191 bit binary field
c2pnb208w1: X9.62 curve over a 208 bit binary field
c2tnb239v1: X9.62 curve over a 239 bit binary field
c2tnb239v2: X9.62 curve over a 239 bit binary field
c2tnb239v3: X9.62 curve over a 239 bit binary field

```

El listado es muy amplio, escogemos por ejemplo secp112r1:

```
1 openssl ecparam -name secp112r1 -out params
```

2. Los usuarios deben generar sus claves privada y pública:

```
1 openssl ecparam -in params -genkey -out clave1
2 openssl ecparam -in params -genkey -out clave2
```

```
eps@labvirteps:~/Escritorio/Cripto/p3/pruebasEC$ openssl pkey -in clave1 -text -noout
Private-Key: (112 bit)
priv:
    00:a9:5c:1d:f2:e4:78:62:1e:1c:39:f9:7f:20:ed
pub:
    04:d0:50:e8:7e:1e:09:e5:4c:c5:92:fd:05:e3:7d:
    4c:81:2b:8c:b0:cd:aa:2d:43:d6:75:60:d6:45
ASN1 OID: secp112r1
```

3. Extraen las claves públicas y las intercambian:

```
1 openssl pkey -in clave1 -pubout -out clave1_pub
2 openssl pkey -in clave2 -pubout -out clave2_pub
```

4. Inferen el secreto común a partir de su clave privada y la clave pública del otro usuario: Usuario1:

```
1 openssl pkeyutl -derive -inkey clave1 -peerkey clave2_pub -out secreto1
```

Usuario2:

```
1 openssl pkeyutl -derive -inkey clave2 -peerkey clave1_pub -out secreto2
```

De nuevo vemos que ambos ficheros tienen el mismo contenido:

```
eps@labvirteps:~/Escritorio/Cripto/p3/pruebasEC$ xxd secreto1
00000000: 89d8 3de5 cc9f 01f1 0c71 9501 9e9d      ..=.....q....
eps@labvirteps:~/Escritorio/Cripto/p3/pruebasEC$ xxd secreto2
00000000: 89d8 3de5 cc9f 01f1 0c71 9501 9e9d      ..=.....q....
eps@labvirteps:~/Escritorio/Cripto/p3/pruebasEC$
```

Por tanto podrán enviar mensajes cifrados con este contenido como clave.

Todos los ficheros originales y cifrados se proporcionan en los archivos de la práctica.

1.3 Generación de Claves Privadas y Públicas

Vamos a explicar en más detalle el funcionamiento de la generación de claves con RSA, y el posterior algoritmo de cifrado y descifrado.

1.3.1 Generación de claves

El objetivo de la generación de claves con RSA es obtener un par de claves (pública-privada) con el que poder cifrar mensajes que se desean enviar a través de un canal posiblemente inseguro, con el fin de que posibles terceras personas no puedan acceder a él. Para ello las claves deben seguir un protocolo o mecanismo para poder ser "compartidas" de tal modo que ambas partes tengan los datos necesarios para cifrar o descifrar el fichero, sin permitir que posibles atacantes puedan inferir los datos para descifrar el mensaje.

Para generar este par de claves con las características descritas RSA se basa en las propiedades de la factorización de enteros, del producto de números primos, y de las propiedades de las potencias. Para generar y compartir estas claves se sigue el siguiente proceso:

1. Se establecen 2 números primos "a" y "b" públicos, y se calcula su producto "m". Este valor "m" será el módulo del cifrado.
2. Se calcula:

$$\phi(m) = \phi(a \cdot b) = \phi(a) \cdot \phi(b) = (a - 1) \cdot (b - 1)$$

3. El usuario que será el receptor del mensaje escoge un entero aleatorio "x" que debe cumplir que:

$$x < \phi(m) \text{ y } x \text{ coprimo con } \phi(m)$$

Este valor "x" será la clave privada que el receptor empleará para descifrar el mensaje.

4. Este mismo usuario genera la clave pública "y" que será el inverso multiplicativo de "x" módulo $\phi(m)$, es decir, halla "y" que cumpla:

$$x \cdot y = 1 \pmod{\phi(m)}$$

Y comparte esta clave pública a través de ese canal posiblemente inseguro, para que el emisor la reciba.

Ya tenemos el par de claves privada-pública. El funcionamiento de este algoritmo se basa en que, por las propiedades de la función de Euler un mensaje "t" perteneciente a $\mathbb{Z}_m$, cumple que:

$$t = t^{xy} \pmod{m}$$

1.3.2 Cifrado y Descifrado

Partiendo de la propiedad mencionada para todo "t" perteneciente a $\mathbb{Z}_m$:

$$t = t^{xy} \pmod{m}$$

. Podemos ver que si se cifra ese mensaje "t" simplemente realizando $t_{cif} = t^y \pmod{m}$, lo cual es posible puesto que "y" es la clave pública, y por tanto es conocida por el emisor; el receptor podría descifrarlo fácilmente $t = t_{cif}^x \pmod{m}$, y, como "x" es la clave privada solo conocida por el receptor, no hay posibilidad de que ningún posible atacante descifre el mensaje aún conociendo todos los demás datos.

El único requisito para poder realizar este proceso de cifrado y descifrado, es que el mensaje "t" sea un entero perteneciente a $\mathbb{Z}_m$, por tanto debe realizarse un algoritmo reversible y conocido por ambas partes para transformar el mensaje inicial en un mensaje que cumpla estas características.

Por tanto el proceso de cifrado y descifrado será:

1. El emisor transforma el mensaje a enviar en un mensaje "t" en forma de entero perteneciente a $\mathbb{Z}_m$, siguiendo el algoritmo reversible acordado.
2. Cifra los enteros del mensaje con la clave pública que le ha enviado el receptor siguiendo la fórmula:

$$t_{cif} = t^y \pmod{m}$$

3. Envía el mensaje al receptor a través del canal inseguro.
4. El receptor descifra el mensaje con la clave privada que solo él conoce siguiendo la fórmula:

$$t = t_{cif}^x \pmod{m}$$

5. El receptor realiza el algoritmo inverso al algoritmo reversible acordado para transformar los enteros del mensaje de vuelta al mensaje original.

De este modo se ha realizado una comunicación segura a través de un canal posiblemente inseguro.

1.3.3 Generacion de claves con OpenSSL

Vamos a crear un par de claves privada-pública con RSA empleando OpenSSL, para ello seguimos los siguientes pasos.

1. Generamos el par de claves de 2048 bits en un fichero "clave", estas claves deben estar cifradas de algún modo, para ello empleamos un algoritmo de cifrado simétrico, en este caso aes con claves de 128 bits:

```
1 openssl genrsa -aes128 -out clave
```

La clave RSA tendrá el siguiente aspecto:



2. Exportamos la clave pública a otro fichero "clave_pub" con el comando:

```
1 openssl rsa -in clave -pubout -out clave_pub
```

```

|-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAzGnLcxQmbdka1qtin3Uf
ctqgTUYL0evYclDB8UJbBDmtjGYxOhRGg2M7uktZ5SuS3DmPPQmf2KCIj0x+sPXh
3ELGKRRmrRvHwmNG7hcHQ3jiVNY9/tpFTYEhE19gn6+p0AvwwcSfkjf5gghBIRgv
iX2qdNJkj4d4lhYhAK1vMnvNnz1Lq14deAtT+sBCaWVsM1MGsWefZ2HPRdta4U0xC
vnzaTPBwBM8eKjouQtIP5wZL3R2+3mFEFoJsoZAD3K1BDFDWxunq4mPq6Kiux0PE
La9a2Gms/6A+bDAsk9FTRfFvXoR1Z9f/PIEescobswsT/ZC0CL05p+AT9d1c0A71
BwIDAQAB
-----END PUBLIC KEY-----

```

3. Se envía esta clave al emisor, que cifrará el mensaje con ella. Típicamente no se cifra el mensaje directamente, sino la clave de un cifrado simétrico empleado para cifrar el mensaje completo, y se manda al receptor el mensaje cifrado con el cifrado simétrico, y la clave simétrica cifrada con su clave pública. De este modo el receptor descifra con su clave privada la clave simétrica, y emplea esta para descifrar el mensaje. En cualquier caso para cifrar con la clave pública empleamos el siguiente comando:

```
1 openssl rsautl -encrypt -inkey clave_pub -pubin -in plaintext -out ciphertxt
```

El fichero "plaintext" en este caso es un fichero de texto con la frase "Este es el mensaje a cifrar". Y el fichero "ciphertxt" resultado es:

```

eps@labvirtips:~/Escritorio/Cripto/p3/pruebasRSA$ xxd ciphertxt
00000000: 3b5a 95c0 703e 4fae 263e fbfd 2c91 8ab5 ;Z..p>0.&>...
00000010: fa0c 31ba 0e5e 746f 6fbe 4cf7 aba2 6711 ..1..^too.L...g.
00000020: c7f1 a57f 0791 1aa4 5d57 e739 3f79 89bf .....]W.9?y..
00000030: ff4b 0e5f 2862 142f 17dd 98b1 068f 013e .K._(b./.....>
00000040: d0ec 3ffa 28c7 a045 dca3 c20f 199c 5fab ..?.(.E....._
00000050: c531 5a5f fb3d 2003 4297 9abd d701 1424 .1Z_. = .B.....$
00000060: 0033 012a 2809 fbfc 2bcd 55ba 934d 4f25 .3.*( ...+.U..MO%
00000070: ea2f 7506 1504 1626 8abe deff 14f0 28cb ./u....&.....(
00000080: 470a cfd3 85ae d7af 7a22 c617 8e34 df7d G.....z"...4.}
00000090: 8693 1446 d169 1e32 285c 60ea 13b8 9748 ...F.i.2(\`....H
000000a0: 858a 572d 41f1 6ded ba4a f3f6 5a9e 9da5 ..W-A.m..J..Z...
000000b0: 36b4 614b 79fa 9357 f773 a515 649a 141c 6.aKy..W.s..d...
000000c0: 3608 a086 da7c e6c3 56fa 2260 9ca8 647c 6....|..V."`.d|
000000d0: 9c60 5832 e101 aaf7 0f12 4e4c cf67 a719 .`X2.....NL.g..
000000e0: 03ad fb87 2d2d 75d1 9a13 9f12 c84c 7f0d .....-u.....L..
000000f0: d317 973c 38ba 3851 66d3 dbfc 33c4 3d4f ...<8.8Qf...3.=0

```

4. Por último el receptor descifrá este mensaje con su clave privada, con el siguiente comando:

```
1 openssl rsautl -decrypt -inkey clave -in ciphertxt -out decrypted_text
```

Obteniendo así en el fichero de salida "decrypted_text" el fichero original.

Todos los ficheros originales y cifrados se proporcionan en los archivos de la práctica.

1.4 Diferencias entre Velocidades de Cifrados Simétricos y Asimétricos

Para comparar las velocidades de cifrado de los cifrados simétricos y asimétricos vamos a calcular la velocidad de diversos cifrados de cada tipo en bits/s, para después compararlos entre sí.

Para los cifrados simétricos vamos a tomar como ejmplos AES, DES, Blowfish, Camellia, y RC2. Para los cifrados asimétricos vamos a tomar como ejmplos RSA, DSA, ECDH y ECDSA.

Observaremos las velocidades de cifrado de cada uno con el comando:

```
1 openssl speed [cifrador]
```

Sustituyendo "[cifrador]" por el tipo de cifrado que vayamos a medir en ese momento.

1.4.1 Simétricos

El comando "speed", en los cifrados simétricos, calcula el número de bloques que cifra el algoritmo en 3 segundos, por tanto, para poder calcular la velocidad en bits/segundo y poder comparar respecto a este valor, debemos dividir entre 3 segundos y multiplicar por el número de bits/bloque.

- AES:

type	16 bytes	64 bytes	256 bytes	1024 bytes	8192 bytes
aes-128 cbc	146296.43k	153526.54k	159739.56k	317999.10k	332247.25k
aes-192 cbc	124324.30k	129664.36k	136177.43k	274047.32k	278684.17k
aes-256 cbc	105651.47k	111016.43k	114499.75k	235028.55k	235427.16k

A la vista de los datos, calculamos los bits/s de cada una de las ejecuciones, y para comparar con el resto de los algoritmos calculamos la media entre los distintos tamaños de bloque. De este modo obtenemos 3 valores:

Media bits/s aes-128 cbc = 206.697.350.000 bits/s

Media bits/s aes-192 cbc = 173.916.950.000 bits/s

Media bits/s aes-256 cbc = 147.159.730.000 bits/s

- DES:

type	16 bytes	64 bytes	256 bytes	1024 bytes	8192 bytes
des cbc	72640.32k	75919.99k	76597.50k	76003.06k	75814.23k
des ede3	27758.34k	28450.77k	28755.63k	28952.14k	28830.38k

Realizamos el mismo proceso que con AES, y, en este caso, obtenemos 2 valores:

Media bits/s des cbc = 48.301.826.005 bits/s

Media bits/s des ede3 = 18.363.459.221 bits/s

- Blowfish:

type	16 bytes	64 bytes	256 bytes	1024 bytes	8192 bytes
blowfish cbc	125160.21k	132453.80k	135374.17k	135156.74k	135495.68k

En el caso de Blowfish obtenemos tan solo 1 valor:

Media bits/s blowfish cbc = 86.234.433.760 bits/s

- Camellia:

type	16 bytes	64 bytes	256 bytes	1024 bytes	8192 bytes
camellia-128 cbc	152988.84k	183481.90k	191822.17k	193208.25k	193110.02k
camellia-192 cbc	120980.96k	137907.75k	143174.44k	144744.97k	146434.05k
camellia-256 cbc	120150.20k	138411.05k	143544.66k	144754.90k	144840.02k

Para Camellia de nuevo obtenemos 3 valores pues está disponible con 3 longitudes de clave:

Media bits/s camellia-128 cbc = 122.873.311.360 bits/s

Media bits/s camellia-192 cbc = 93.014.735.659 bits/s

Media bits/s camellia-256 cbc = 92.156.814.848 bits/s

- RC2:

type	16 bytes	64 bytes	256 bytes	1024 bytes	8192 bytes
rc2 cbc	47462.85k	48405.70k	49649.36k	49939.93k	49699.96k

Por último en el caso de RC2 obtenemos de nuevo tan solo 1 valor:

Media bits/s RC2 cbc = 31.656.544.480 bits/s

Observando los resultados de todos estos algoritmos de cifrado simétrico, podemos elaborar la siguiente tabla:

	AES			DES		Blowfish	Camellia			RC2
	128 CBC	192 CBC	256 CBC	CBC	EDE	CBC	128 CBC	192 CBC	256 CBC	CBC
Velocidad (bits/s)	206.697.350.000	173.916.930.000	147.159.730.000	48.301.826.005	18.368.439.221	86.234.433.790	122.873.311.360	98.014.735.659	92.136.814.848	31.656.544.480

Entre los cifrados simétricos medidos, el más veloz es AES, esto se debe ya que, aunque el cálculo de las SBOXES es complejo, una vez estas están calculadas el cifrado y descifrado son bastante sencillos, y muy veloces. Por su parte el más lento (excluyendo DES por emplear otro modo) es el algoritmo RC2, lo cual tiene sentido pues tiene una estructura de operación compleja al emplear un Feistel no balanceado. Sorprende que DES siendo un algoritmo muy rápido, no se encuentre entre los más rápidos, siendo, de hecho uno de los más lentos de entre los evaluados.

1.4.2 Asimétricos

Para los cifrados asimétricos, el comando "speed" calcula los bloques de bits cifrados o firmados en un período de 10 segundos. por tanto, el proceso a realizar para calcular su velocidad en bits/s será el mismo que para los simétricos, pero dividiendo entre 10 en lugar de 3.

- RSA:

```
Doing 512 bit private rsa's for 10s: 226434 512 bit private RSA's in 9.96s
Doing 512 bit public rsa's for 10s: 3572178 512 bit public RSA's in 9.96s
Doing 1024 bit private rsa's for 10s: 77174 1024 bit private RSA's in 9.99s
Doing 1024 bit public rsa's for 10s: 1354127 1024 bit public RSA's in 9.99s
Doing 2048 bit private rsa's for 10s: 12078 2048 bit private RSA's in 9.98s
Doing 2048 bit public rsa's for 10s: 407381 2048 bit public RSA's in 9.99s
Doing 4096 bit private rsa's for 10s: 1695 4096 bit private RSA's in 9.97s
Doing 4096 bit public rsa's for 10s: 110583 4096 bit public RSA's in 9.97s
```

Vamos a dividir los datos obtenidos en 2 grupos, los de cálculo de la clave privada, y los de la pública.

De este modo obtenemos los valores:

Media bits/s RSA priv = 5.665.971 bits/s

Media bits/s RSA pub = 112.571.136 bits/s

- DSA:

```
Doing 512 bit sign dsa's for 10s: 226272 512 bit DSA signs in 9.92s
Doing 512 bit verify dsa's for 10s: 254019 512 bit DSA verify in 9.98s
Doing 1024 bit sign dsa's for 10s: 104111 1024 bit DSA signs in 9.98s
Doing 1024 bit verify dsa's for 10s: 104254 1024 bit DSA verify in 9.99s
Doing 2048 bit sign dsa's for 10s: 34487 2048 bit DSA signs in 9.97s
Doing 2048 bit verify dsa's for 10s: 31627 2048 bit DSA verify in 9.98s
```

En DSA distinguimos de nuevo 2 casos, los tiempos de firma, y los de verificación:

Media bits/s DSA sign = 9.769.677 bits/s

Media bits/s DSA verify = 10.052.864 bits/s

- ECDH:

```

Doing 160 bit ecdh's for 10s: 61048 160-bit ECDH ops in 9.97s
Doing 192 bit ecdh's for 10s: 50262 192-bit ECDH ops in 9.99s
Doing 224 bit ecdh's for 10s: 37081 224-bit ECDH ops in 9.98s
Doing 256 bit ecdh's for 10s: 162495 256-bit ECDH ops in 9.99s
Doing 384 bit ecdh's for 10s: 15514 384-bit ECDH ops in 9.98s
Doing 521 bit ecdh's for 10s: 6780 521-bit ECDH ops in 9.97s
Doing 163 bit ecdh's for 10s: 49628 163-bit ECDH ops in 9.97s
Doing 233 bit ecdh's for 10s: 37608 233-bit ECDH ops in 9.98s
Doing 283 bit ecdh's for 10s: 20422 283-bit ECDH ops in 9.98s
Doing 409 bit ecdh's for 10s: 12222 409-bit ECDH ops in 9.98s
Doing 571 bit ecdh's for 10s: 5589 571-bit ECDH ops in 9.98s
Doing 163 bit ecdh's for 10s: 49252 163-bit ECDH ops in 9.99s
Doing 233 bit ecdh's for 10s: 36800 233-bit ECDH ops in 9.98s
Doing 283 bit ecdh's for 10s: 19741 283-bit ECDH ops in 9.99s
Doing 409 bit ecdh's for 10s: 11551 409-bit ECDH ops in 9.98s
Doing 571 bit ecdh's for 10s: 5230 571-bit ECDH ops in 10.00s

```

En este caso tan solo obtenemos un valor:

Media bits/s ECDH = 872.088 bits/s

- ECDSA:

```

Doing 160 bit sign ecdsa's for 10s: 190583 160 bit ECDSA signs in 9.78s
Doing 160 bit verify ecdsa's for 10s: 50496 160 bit ECDSA verify in 9.96s
Doing 192 bit sign ecdsa's for 10s: 167543 192 bit ECDSA signs in 9.93s
Doing 192 bit verify ecdsa's for 10s: 43204 192 bit ECDSA verify in 9.98s
Doing 224 bit sign ecdsa's for 10s: 125111 224 bit ECDSA signs in 9.98s
Doing 224 bit verify ecdsa's for 10s: 31243 224 bit ECDSA verify in 9.99s
Doing 256 bit sign ecdsa's for 10s: 264312 256 bit ECDSA signs in 9.96s
Doing 256 bit verify ecdsa's for 10s: 110513 256 bit ECDSA verify in 9.99s
Doing 384 bit sign ecdsa's for 10s: 54752 384 bit ECDSA signs in 9.98s
Doing 384 bit verify ecdsa's for 10s: 12733 384 bit ECDSA verify in 9.99s
Doing 521 bit sign ecdsa's for 10s: 24574 521 bit ECDSA signs in 9.99s
Doing 521 bit verify ecdsa's for 10s: 5746 521 bit ECDSA verify in 10.00s
Doing 163 bit sign ecdsa's for 10s: 57245 163 bit ECDSA signs in 9.97s
Doing 163 bit verify ecdsa's for 10s: 27878 163 bit ECDSA verify in 9.99s
Doing 233 bit sign ecdsa's for 10s: 29709 233 bit ECDSA signs in 9.99s
Doing 233 bit verify ecdsa's for 10s: 22570 233 bit ECDSA verify in 9.97s
Doing 283 bit sign ecdsa's for 10s: 19911 283 bit ECDSA signs in 9.98s
Doing 283 bit verify ecdsa's for 10s: 11852 283 bit ECDSA verify in 9.99s
Doing 409 bit sign ecdsa's for 10s: 8874 409 bit ECDSA signs in 9.98s
Doing 409 bit verify ecdsa's for 10s: 7259 409 bit ECDSA verify in 9.99s
Doing 571 bit sign ecdsa's for 10s: 4124 571 bit ECDSA signs in 9.99s
Doing 571 bit verify ecdsa's for 10s: 3288 571 bit ECDSA verify in 9.99s
Doing 163 bit sign ecdsa's for 10s: 57194 163 bit ECDSA signs in 9.99s
Doing 163 bit verify ecdsa's for 10s: 27316 163 bit ECDSA verify in 9.99s
Doing 233 bit sign ecdsa's for 10s: 29958 233 bit ECDSA signs in 9.99s
Doing 233 bit verify ecdsa's for 10s: 21753 233 bit ECDSA verify in 9.99s
Doing 283 bit sign ecdsa's for 10s: 20043 283 bit ECDSA signs in 9.99s
Doing 283 bit verify ecdsa's for 10s: 11191 283 bit ECDSA verify in 9.99s
Doing 409 bit sign ecdsa's for 10s: 8856 409 bit ECDSA signs in 9.99s
Doing 409 bit verify ecdsa's for 10s: 6851 409 bit ECDSA verify in 10.00s
Doing 571 bit sign ecdsa's for 10s: 4121 571 bit ECDSA signs in 9.99s
Doing 571 bit verify ecdsa's for 10s: 3044 571 bit ECDSA verify in 9.99s

```

De nuevo como en DSA, distinguimos los casos de firma y verificación:

Media bits/s ECDSA sign = 1.550.003 bits/s
 Media bits/s ECDSA verify = 592.369 bits/s

A partir de estos resultados generamos de nuevo una tabla comparativa:

	RSA		DSA		ECDH	ECDSA	
	Priv	Pub	Sign	Verif		Sign	Verif
Velocidad (bits/s)	5.665.971	112.571.136	9.769.677	10.052.864	872.088	1.550.003	592.369

En esta tabla podemos observar la tremenda diferencia de velocidad que existe en RSA cuando calcula la clave privada, frente a cuando calcula la pública, esto se debe a que el algoritmo empleado por RSA para generar el número pseudoaleatorio que dará lugar a la clave privada es bastante complejo, mientras que el cálculo a realizar sobre esta clave privada para obtener la pública es mucho más sencillo. Por tanto, si tomamos el algoritmo RSA como el cálculo de clave privada, pues es donde radica la complejidad de este, vemos claramente que el algoritmo más veloz es DSA. Esto tiene mucho sentido, pues es un algoritmo que se limita a firmar el fichero para proporcionar una autenticación, pero no genera ningún tipo de claves pública-privada, que es el proceso más costoso. Por el contrario los más lentos son los que emplean curvas elípticas para el cálculo de su par de claves, pues las operaciones a realizar con estas son significativamente más complejas.

1.4.3 Comparación

A la vista de los resultados obtenidos, podemos ver que la velocidad de los cifrados simétricos es enormemente mayor que el de los asimétricos. Tanta es la diferencia, que si tomamos la velocidad del algoritmo simétrico más lento de los evaluados (DES modo EDE3), y el más rápido de los asimétricos (RSA en el cálculo de la clave pública, que ni siquiera es representativo de la complejidad de RSA), vemos que este DES es 163 veces más rápido. Si en lugar de este fragmento de RSA, tomamos una velocidad realmente significativa que nos de una comparación realista (en este caso el más veloz es DSA al verificar la firma) esta diferencia aumenta aún más, siendo en este caso DES 1827 veces más rápido.

Esta diferencia tan abrumadora de velocidad es el motivo principal por el que los algoritmos de cifrado asimétrico no se emplean, por lo general, para cifrar mensajes completos, sino únicamente para obtener unas claves con las que generar un canal seguro para enviar el mensaje, siendo este cifrado por un algoritmo simétrico mucho más rápido.

De este modo, podemos obtener un modo seguro de comunicación sin que el coste elevado del algoritmo asimétrico suponga un tiempo extremadamente alto, ya que solo se empleará sobre un par de claves de tamaño muy pequeño en comparación con el mensaje, que se cifra con un algoritmo simétrico mucho más veloz.

1.5 Certificados X.509

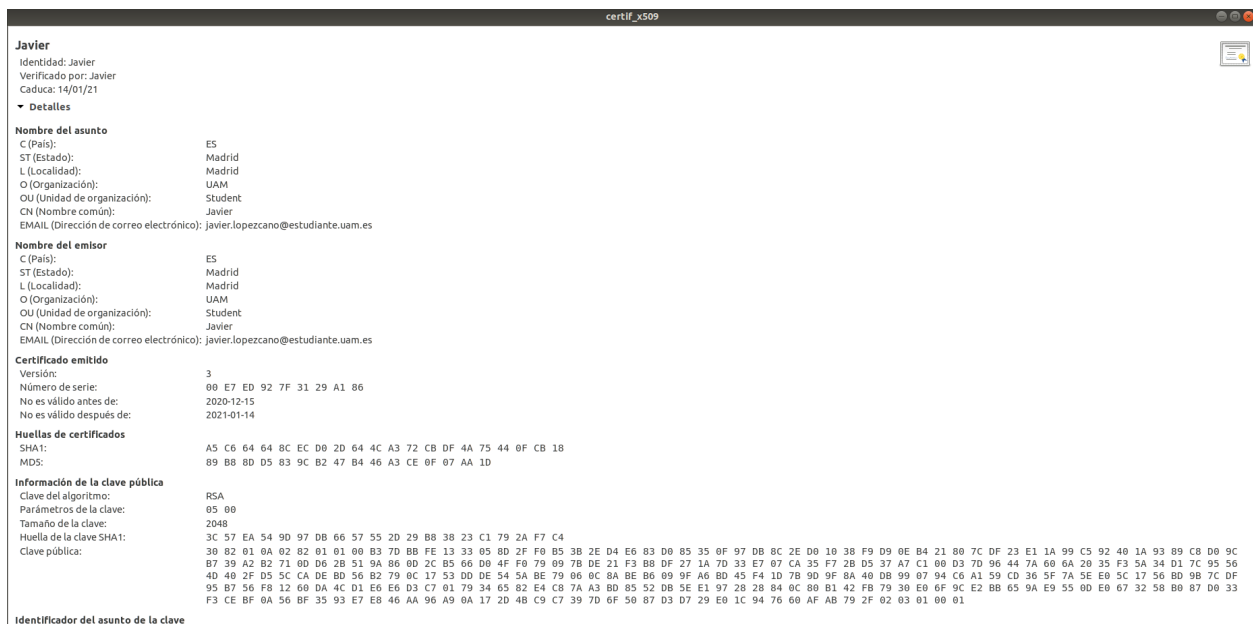
Un certificado X.509 es un certificado digital acreditado por una "entidad acreditadora" para verificar la identidad de la persona a la que pertenece una firma pública. Cuando una persona emplea una firma pública para cifrar un mensaje mediante un cifrado asimétrico, asume que esa clave pública corresponde a una determinada persona que tendrá la clave privada, sin embargo, no hay forma segura de confirmar que la firma pertenece realmente a la persona adecuada. Precisamente esta función es la que cumplen los certificados X.509. Son certificados en los que una entidad verifica o acredita que una determinada clave pública es propiedad de un usuario. En estos certificados se especifica además del usuario y la clave pública acreditada, la entidad certificadora y su firma, así como los datos tanto de la entidad certificadora como de la persona a la que pertenece la clave pública. De este modo, si la entidad certificadora que acredita la clave mediante este certificado, es una entidad conocida, de prestigio y de confianza, proporciona una seguridad y unas garantías al emisor, de que la clave con la que va a cifrar corresponde efectivamente al usuario receptor adecuado. Por tanto, en la mayoría de comunicaciones oficiales a través de un mensaje cifrado con un algoritmo asimétrico de clave pública-privada como RSA, la empresa oficial (pública o privada) que va a emitir el mensaje, exige que la clave pública del usuario esté acreditada por alguna entidad certificadora de confianza como SSL, que les garantice que el receptor es realmente quien dice ser.

OpenSSL nos permite generar certificados X.509 para poder acreditarnos como usuario de una clave pública, sin embargo para que este certificado sirva como acreditación para la mayoría de comunicaciones oficiales, sería necesario conseguir la firma de una entidad de confianza para firmarlo. Como no tenemos acceso a esta, y no vamos a solicitarla, vamos a probar a generar un certificado firmado por nosotros mismos (auto-firmado), para ver el funcionamiento del comando de creación de estos certificados en OpenSSL, y estudiar la estructura de estos.

Generamos un nuevo certificado X.509 auto-firmado empleando el comando "req" proporcionado por OpenSSL. Para ello ejecutamos la siguiente sentencia en el terminal:

```
1 openssl req -new -x509 -newkey rsa:2048 -keyout clave -out certif_x509
```

Este comando solicita a OpenSSL un nuevo certificado X.509 que se genera en el fichero "certif_x509", firmado con una firma RSA de 2048 bits generada al momento, que se almacenará en el fichero "clave". Al ejecutar el comando, se nos solicitarán varios datos como la contraseña para la generación de las claves RSA, y el nombre, país, ciudad, etc necesarios para completar el certificado. Tras finalizar, se generará el fichero "certif_x509" que contiene:



También es posible consultar los datos del certificado con OpenSSL a través del comando "x509", de este modo podemos consultar:

- Número de serie:

```
1 openssl x509 -in certif_x509 -serial
```

- La clave pública que otorga el certificado:

```
1 openssl x509 -in certif_x509 -pubkey
```

- El Hash de la firma tanto del usuario como de la entidad certificadora (nosotros mismos en este caso):

```
1 openssl x509 -in certif_x509 -subject_hash -issuer_hash
```

- Los nombres del usuario y la entidad certificadora:

```
1 openssl x509 -in certif_x509 -subject -issuer
```

- Correo electrónico:

```
1 openssl x509 -in certif_x509 -email
```

- Fecha de inicio y fin de validez:

```
1 openssl x509 -in certif_x509 -dates
```

Vamos a solicitar todos estos datos a través de un comando con todas las flags:

```

1 openssl x509 -in certif_x509 -serial -subject -issuer -subject_hash -issuer_hash -serial_email -email -dates -pubkey

-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQFAAQCAQ8AMIIBCgKCAQEA327/hmZBY0v8LU7LTm
g9CFNq+X24wu0BA4+dk0tCGAFN8j4qZxZAGp0jNcctzmlsnEN1tRmoVNLvM
0E/weQl73lHzuN8nGn0z5wfKNfcr1TenwQDTFZZEebBqIDxwJTRfJVWUAv1VzK
3r1WsnKF1Pd3lRavnkGDIq+tgmpfr1F9B17nZ+KQNUZB5TGoVnN196XuBcF1a9
m3zflbdw+B3g2kzR5ubTxw5fNGWCSMh6o72Futte4ZcoKIQMGfLC+3kw4G+c4rtl
mulVDeBnMLwh9Az886/Cl/NZPn6Egq1KqkFy1Ycc5f9hQ9PXKEAcLHzgr6t5
LwIDAQAB
-----END PUBLIC KEY-----
-----BEGIN CERTIFICATE-----
MIID+zCCAuoGAWIBAgIJA0ftkn8xKaGMA0GCSqGSIb3DQEBCwUAMIGTMsQcQYD
QWQGEwJFuzEPMA0GA1UECwGtWfKcm1kM08wDQYDVQQHDAZNYWRYaWwQXDAKBNV
BAOMA1VBTTQMA4GA1UECwwHU3R1ZG9udDEPMA0GA1UEAwGSmF2aHVyYmVtLWwYJ
KoZIHvcNAQkBF1JqYXZpZXIubG9wZXBjYV5vQGVzZHVhKwFudGUudWfLtnVzMB4X
DTIwMTIxNTEzMzMwM1oDTIwMDEzMDZzMzMwM1owZDZmXCAzBgNVBAYTAkVMTQ8w
DQYDVQQIDAZNYWRYaWwQXDAzANBgNVBAcMBk1hZjZpZDEMAA0GA1UECgwDVUFNRMAW
DgYDVQQLDAdHvKZ5M08wDQYDVQQDAAZKYXZpZXIwTAVBgkqhkiG9w0BCQEW
Imphdmllnc15s3b3lemlhbm9AZXN0dHRpY50ZS51YW0uZXMwggE1MA0GCSqGSIb3
DQEBAQUAA4IBDwAwggEKAoIBAQCzfbv+EzMFjS/wtTsu10ad0IU1D5fbjC7QEDj5
2Q60fYB83yPhGpnFkkaAk4nI0Jy30akYc3QWk1Gahg0stwbQ7/B5CXveIF043yca
FTpNB80i19yvVn6fBANN9lkr6YgogNfnaNf8lVZNQC/VXMrevVayeQwXU93eVfq+
eQYm1r62Cz+mvUOXHudn4pa25kHlMahic82X3pe4FwXvr2bfN+Vt1b4EnDaTNHm
5TPHAXk0ZYLKyHqjYVVS217hlygohAYASULrTDGbsz1u2W6VUN4GcyLlCH0DPZ
zr8KVR81k+forqqWQ0xLUvJxz19b1CH09cp4ByUdmCvq3kAgVMBAAQJUDBOMB0G
A1UddgQWBBRZPPIEMdcuUvMRL2jP/IVdbrC+zAFBgNVHSMGDAwBRZPPIEMdcu
uUvMRL2jP/IVdbrC+zAMBgnVHRMEBTADAQAQ/MA0GCSqGSIb3DQEBCwUAA4IBAQBf
HYFPw95py8dI2+UaqIEFsb1wCz+Puk1DhVeqgYALHwS+ZgoK2Rl+RW1ntKL6vAys
LRMvQhpkn2NBfAmxyXJUn9Eo1CTZNEHQ4sTp+8z1H8XxG62pTd1+AYeFBEtHIULt
00/r5KAZVP0ESXMSH24LXZhtdmZew7fMnk3xu4MoZu+s8XWQSPRfhu0p9n4An3d
1UvRLzR4AZMStnx+Lw2a0144hwMtncBxjE7dMC8zD9hLcwa3lQG8B2haaXDKkvM
L7X+VzCq5D8ULGT4ndTRVyFR06Cms9CBHP9twIJaagJcKulwbHPnVn6MzLgntB8
O+y6Rd0Y/1L+FoTIyb3k
-----END CERTIFICATE-----

```

Existen también flags para consultar otros datos, como la huella, el propósito del certificado, los propósitos para los que no vale, etc. Es posible también imprimir el certificado completo mediante la flag "-text" con el comando:

```
1 openssl x509 -in certif_x509 -text
```

De este modo obtenemos la siguiente salida:

```

1  eps@labvirt:~/Escritorio/Cripto/p3/X509$ openssl x509 -in certif_x509 -text
2  Certificate:
3      Data:
4          Version: 3 (0x2)
5          Serial Number:
6              e7:ed:92:7f:31:29:a1:86
7          Signature Algorithm: sha256WithRSAEncryption
8          Issuer: C=ES, ST=Madrid, L=Madrid, O=UAM, OU=Student, CN=Javier/emailAddress=javier.lopezcano@estudiante.uam.es
9          Validity
10             Not Before: Dec 15 13:33:03 2020 GMT
11             Not After : Jan 14 13:33:03 2021 GMT
12             Subject: C=ES, ST=Madrid, L=Madrid, O=UAM, OU=Student, CN=Javier/emailAddress=javier.lopezcano@estudiante.uam.es
13             Subject Public Key Info:
14                 Public Key Algorithm: rsaEncryption
15                 Public-Key: (2048 bit)
16                 Modulus:
17                     00:b3:7d:bb:fe:13:33:05:8d:2f:f0:b5:3b:2e:d4:
18                     e6:83:d0:85:35:0f:97:db:8c:2e:d0:10:38:f9:d9:
19                     0e:b4:21:80:7c:df:23:e1:1a:99:c5:92:40:1a:93:
20                     89:c8:d0:9c:b7:39:a2:b2:71:0d:d6:2b:51:9a:86:
21                     0d:2c:b5:66:d0:4f:f0:79:09:7b:de:21:f3:b8:df:
22                     27:1a:7d:33:e7:07:ca:35:f7:2b:d5:37:a7:c1:00:
23                     d3:7d:96:44:7a:60:6a:20:35:f3:5a:34:d1:7c:95:
24                     56:4d:40:2f:d5:5c:ca:de:bd:56:b2:79:0c:17:53:
25                     dd:de:54:5a:be:79:06:0c:8a:be:b6:09:9f:a6:bd:
26                     45:f4:1d:7b:9d:9f:8a:40:db:99:07:94:c6:a1:59:
27                     cd:36:5f:7a:5e:e0:5c:17:56:bd:9b:7c:df:95:b7:
28                     56:f8:12:60:da:4c:d1:e6:e6:d3:c7:01:79:34:65:

```


2 RSA

Para hacer este ejercicio nos ha sido útil consultar algunos libros de la bibliografía: para la potenciación modular y el algoritmo Miller-Rabin para generar números primos hemos usado el *Handbook of Applied Cryptography*; para factorizar el módulo de RSA conociendo el exponente de descifrado, la sección *Other Attacks on RSA* del *Cryptography: Theory and Practice*.

2.1 Potenciación de grandes números

Para esta sección decidimos implementar los algoritmos **14.79** y **14.82** del *Handbook of Applied Cryptography*. El primero se conoce como exponenciación binaria de izquierda a derecha (función `lr_bin_exp`), mientras que el segundo es una generalización del primero añadiendo una fase previa de precomputación, y se le conoce como exponenciación k -aria de izquierda a derecha (función `lr_kary_exp`). Para obtener la representación en la base adecuada del exponente e hemos utilizado la función `mpz_get_str`. Por limitaciones de esta función, la exponenciación k -aria solo se puede hacer para $k = 1, 2, 3, 4, 5$.

Para el programa `potencia` usamos la exponenciación k -aria con $k := 5$. Decidimos usar este algoritmo al ver que en CPython, la implementación en C de Python habitual, se usa este mismo algoritmo para el mismo valor de k para hacer exponenciación modular de números grandes.

Incluimos los pseudocódigos del libro que hemos implementado, así como una gráfica comparativa del rendimiento de nuestras implementaciones con la de `gmp`. Se puede observar como la mejor de nuestras implementaciones es algo menos eficiente que la de `gmp`. Otro detalle interesante es que la exponenciación binaria es ligeramente más eficiente que la exponenciación k -aria con $k := 1$, a pesar de ser equivalentes. Esto se debe a esa primera fase de precomputación no supone una mejora lo bastante notable para compensar su coste.

Nota. los resultados de la comparativa de rendimientos son especialmente sensibles al tamaño de los números aleatorios con el que se hacen las operaciones. Los datos han sido obtenidos con números aleatorios de 4096 bits. Empíricamente, hemos observado que para números de menos bits la implementación de `gmp` es bastante mejor que la nuestra, pero conforme aumentamos el número de bits recortamos distancia.

Rendimiento Comparado

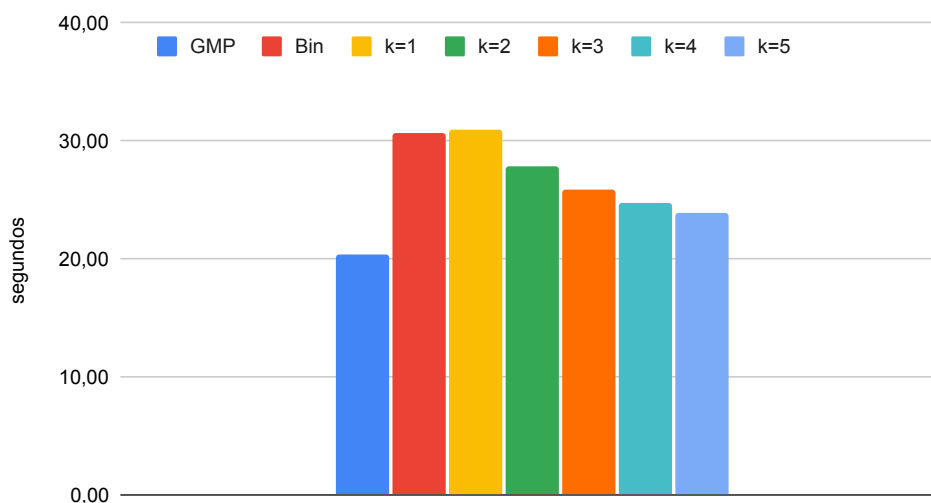


Figure 1: Rendimiento comparado de las distintas implementaciones

14.79 Algorithm Left-to-right binary exponentiation

INPUT: $g \in G$ and a positive integer $e = (e_t e_{t-1} \cdots e_1 e_0)_2$.

OUTPUT: g^e .

1. $A \leftarrow 1$.
 2. For i from t down to 0 do the following:
 - 2.1 $A \leftarrow A \cdot A$.
 - 2.2 If $e_i = 1$, then $A \leftarrow A \cdot g$.
 3. Return(A).
-

Figure 2: Pseudocódigo *Left-right binary exponentiation*

14.82 Algorithm Left-to-right k -ary exponentiation

INPUT: g and $e = (e_t e_{t-1} \cdots e_1 e_0)_b$, where $b = 2^k$ for some $k \geq 1$.

OUTPUT: g^e .

1. *Precomputation.*
 - 1.1 $g_0 \leftarrow 1$.
 - 1.2 For i from 1 to $(2^k - 1)$ do: $g_i \leftarrow g_{i-1} \cdot g$. (Thus, $g_i = g^i$.)
 2. $A \leftarrow 1$.
 3. For i from t down to 0 do the following:
 - 3.1 $A \leftarrow A^{2^k}$.
 - 3.2 $A \leftarrow A \cdot g_{e_i}$.
 4. Return(A).
-

Figure 3: Pseudocódigo *Left-right k -ary exponentiation*

2.1.1 Uso del programa: potencia

El formato del programa es:

```
./potencia base exponente modulo [-v]
```

Los tres primeros parámetros son autoexplicativos y el último es opcional y activa el modo *verboso*. Si la base es g , el exponente e y el módulo m , el programa debería devolver el resultado de hacer la operación

$$g^e \mod m.$$

```
./potencia 127873 837646 564634
252249
```

```
./potencia 127873 837646 564634 -v
-----
g := 127873
e := 837646
m := 564634
g^e mod m = 252249
-----
```

2.1.2 Uso del programa: test_potencia

El formato del programa es:

```
./test_potencia [numero_iteraciones]
```

Este programa mide el rendimiento de las distintas implementaciones de la exponenciación modular y comprueba que funcionan correctamente. Esto lo hace haciendo tantas operaciones como indique el parámetro

`numero_ejecuciones`. Las operaciones se hacen con números aleatorios de 4096 bits, y se hacen 1000 iteraciones en caso de no indicarse por la entrada.

Nota. no se recomienda poner más de 1000 iteraciones. Si se modifica este parámetro, que se haga a la baja. Las operaciones son con números de 4096 bits y podría tardar más de lo razonable.

```
./test_potencia
----- Tiempo en realizar 1000 potencias
Tiempo funcion de gmp:      20.331280
Tiempo LR binaria:         30.686226
Tiempo LR 1-aria:          30.968544
Tiempo LR 2-aria:          27.833224
Tiempo LR 3-aria:          25.941359
Tiempo LR 4-aria:          24.700374
Tiempo LR 5-aria:          23.901902
-----
Los resultados de las operaciones coinciden para todas funciones
```

2.2 Generación de números primos: Miller-Rabin

Hemos implementado el algoritmo de Miller-Rabin tal y como aparece descrito en *Handbook of Applied Cryptography*. Para generar primos aleatorios con Miller-Rabin usamos una versión simplificada del algoritmo que sale en el mismo libro. Las funciones del código que implementan estas funcionalidades son `mr_test` y `gen_prime_mr`, para aplicar el test de Miller-Rabin para un número y un parámetro de seguridad dados, y generar primos aleatorios dado un número de bits y una probabilidad de error.

Una particularidad de cómo funciona nuestro programa es que recibe como parámetro de seguridad un número positivo s , que se trata del número al que tenemos elevar $1/10$ para obtener la probabilidad de error deseada. Cuanto mayor el s , menor es la probabilidad de error por al menos un orden de magnitud.

Cuando ejecutamos el programa con una probabilidad de error $p = 10^{-s}$ y un número de bits n , tenemos el problema de encontrar el número de iteraciones de Miller-Rabin que tenemos que aplicar para obtener la seguridad deseada. Para ello, utilizamos la expresión de la probabilidad de error cuando $n \gg 1$ y resolvemos para el menor m la siguiente inecuación:

$$P(\text{fallo de Miller-Rabin}) \approx \frac{1}{1 + \frac{4^m}{n \log 2}} \leq p,$$

despejando m obtenemos

$$\frac{\log \frac{(1-p)n \log 2}{p}}{\log 4} \leq m,$$

y el menor m que cumple esto es el techo de la parte de la izquierda. Por ejemplo, comprobamos que para $s := 20$, $p := 10^{-20}$ y $n := 8192$, nos sale un $m := 40$. Esto significa que para obtener una probabilidad de fallo menor que 10^{-20} generando números de 8192 bits necesitamos al menos 40 iteraciones de Miller-Rabin.

Pregunta del enunciado. Se nos pregunta qué influencia tiene el parámetro p en los resultados, y que probemos a generar un número de 8192 bits. Cuanto menor es la probabilidad de ser un falso primo que queremos que tenga nuestro programa, más va a tardar en ejecutarse, pero mayor será su fiabilidad.

4.24 Algorithm Miller-Rabin probabilistic primality test

MILLER-RABIN(n, t)INPUT: an odd integer $n \geq 3$ and security parameter $t \geq 1$.OUTPUT: an answer “prime” or “composite” to the question: “Is n prime?”

1. Write $n - 1 = 2^s r$ such that r is odd.
 2. For i from 1 to t do the following:
 - 2.1 Choose a random integer a , $2 \leq a \leq n - 2$.
 - 2.2 Compute $y = a^r \bmod n$ using Algorithm 2.143.
 - 2.3 If $y \neq 1$ and $y \neq n - 1$ then do the following:

$j \leftarrow 1$.
While $j \leq s - 1$ and $y \neq n - 1$ do the following:

Compute $y \leftarrow y^2 \bmod n$.
If $y = 1$ then return(“composite”).
 $j \leftarrow j + 1$.
If $y \neq n - 1$ then return(“composite”).
 3. Return(“prime”).
-

Figure 4: Pseudocódigo de Miller-Rabin

4.44 Algorithm Random search for a prime using the Miller-Rabin test

RANDOM-SEARCH(k, t)INPUT: an integer k , and a security parameter t (cf. Note 4.49).OUTPUT: a random k -bit probable prime.

1. Generate an odd k -bit integer n at random.
 2. Use trial division to determine whether n is divisible by any odd prime $\leq B$ (see Note 4.45 for guidance on selecting B). If it is then go to step 1.
 3. If MILLER-RABIN(n, t) (Algorithm 4.24) outputs “prime” then return(n). Otherwise, go to step 1.
-

Figure 5: Pseudocódigo para generar números aleatorios con Miller-Rabin. Nosotros no implementamos el paso 2 del algoritmo

2.2.1 Uso del programa: primo

El formato del programa es:

```
./primo -b numero_bits -p sec [-o fichero_salida]
```

El primer parámetro es el número de bits que queremos que tenga el (pseudo)primo aleatorio que vamos a generar, mientras que el segundo es el entero positivo al que tenemos que elevar $1/10$ para obtener la probabilidad de fallo deseada.

A continuación se muestra el resultado de nuestro programa cuando se le pide generar un número de 8192 bits y una probabilidad de error de 10^{-20} . Como hemos visto antes, el número de iteraciones de Miller-Rabin para satisfacer estas condiciones es 40.

```
./primo -b 8192 -p 20
---- PSEUDOPRIMO ----
22669716793259548436570396702134697005319091183770408588856344837621040993216867496049080329
33689422557738931073537498177862076458222739808245999586327670552756424292728789604231039333
12107017234989252811614867818503442991176071519312916140512386539818964622795382271566099637
08900341436675554837431250108697912731493827731294346800623455336921916773310274634185607426
63204545288160901635308999117098605212209261452097033095913306607486791954605832309729014122
39766537601150590469526325115021632257292552260358845677179440218204580078380358522904669870
06876269319200516681850963524637275352703461531651465953411427570903573690459646843739582513
43311590611939635309773093229673439720078555980849244283008451586229176130594278054508371668
72741561888338187346040103368432921883699414388540339901221350371184536273093598233563999225
27585277285467378132593508252407787808159002926589115528360412325752090912590117913975671755
93648913703706888468112419130286171002939779844774832788407717795901532580041847591058438114
```

```

10305025350338582458361006527702822829768935081531983683104491520030406134283925260956073122
06412350269053948961176211818101238175107064350737876715009498730039964070356607726683726974
72249642688045540736793827297104875216743242683654810674850691361245535857919917973779069742
04337283155117569040774984980869433853243033216530912634655646222868527230694193239875746029
12463757122398503926255321531998290754453577894079870192494270216593601101490865519364194438
15142796084182121262089282745349217854118387482763520046888649069898291690683069586200585603
05778780901561094382095997028692701936457037854270808557233039690142892689070243450051691969
75606257913469548177814454968467196209386460282523986258623986385118059063434980054419982967
45538490866740951632419588165019791797037530861523225961208642229719718616224176984919213441
32220395385914833650949257797650434838022134584395242456132269925309603386576935512752172079
64294144722855308066949619878318739088399951134391644193975189213505415789019634353016029565
38527523263798112381102254440486340975461338056194247954043583517886704091181630770842313550
42154014939533842131678419754517266559483084426002508541672146251796736783316433032739491542
26704954043996373008638996757570727567855826922427373619605493659717354399453457476566434675
29523244358230797112771337986145324947912327186161907782657711022117814353704098948842827019
63596973241831407431767945383619404333197197183173754826887598242068624819
---- FIN PSEUDOPRIMO ----
----- 0=Compuesto;1=PrimoProbable;2=Primo
Nuestro resultado: 1
Resultado de gmp: 1
Numero de repeticiones: 40
Probabilidad error: < 10^-20
Tiempo (s) en generar: 328.377000
-----

```

La información que devuelve es:

- Nuestro resultado. Siempre va a ser 1 porque el número que generamos es primo probable para nuestro algoritmo.
- Resultado de `gmp`. Es el resultado de aplicar la función de `gmp` al número generado con el número de repeticiones que toquen. Puede ser 0, 1 o 2, dependiendo de si es un número compuesto, un primo probable, o un primo con total certeza.
- Número de repeticiones. Autoexplicativo.
- Probabilidad de error. Autoexplicativo.
- Tiempo en generar. Tiempo en segundos que se ha tardado en generar el primo.

Otro ejemplo de ejecución del programa usando el fichero de salida.

```

./primo -b 1024 -p 20 -o pgen
cat pgen
---- PSEUDOPRIMO ----
14558030461338200019210715368132075787388468456083362370776002191970500118964238530845267961
15516394674741078690218642303686614399792073266639363059717183693402654073759477034607150316
54736405600879987203688886817719564225493351148886139081038439600283155215891821336876879383
30495989084663918789658712089587
---- FIN PSEUDOPRIMO ----
----- 0=Compuesto;1=PrimoProbable;2=Primo
Nuestro resultado: 1
Resultado de gmp: 1
Numero de repeticiones: 38
Probabilidad error: < 10^-20
Tiempo (s) en generar: 0.168852
-----

```

2.3 Factorización del módulo de RSA mediante el conocimiento de exponente de descifrado

Se nos pide implementar un algoritmo que factorice rápidamente el módulo de RSA conociendo el exponente de descifrado. Poco más que decir en esta sección, a no ser que expliquemos en que consiste en algoritmo para realizar este ataque a RSA, que no lo haremos porque ya se ha hecho en clase. El pseudocódigo en concreto que hemos seguido para implementar el algoritmo lo hemos obtenido del Stinson, e incluimos una imagen a continuación. La función que implementa esto se llama `break_rsa_miller`. El Miller es porque en otras fuentes hemos visto este algoritmo citado como Algoritmo de Miller.

Algorithm 6.10: RSA-FACTOR(n, a, b)**comment:** we are assuming that $ab \equiv 1 \pmod{\phi(n)}$ write $ab - 1 = 2^s r, r$ oddchoose w at random such that $1 \leq w \leq n - 1$ $x \leftarrow \gcd(w, n)$ **if** $1 < x < n$ **then return** (x)**comment:** x is a factor of n $v \leftarrow w^r \pmod{n}$ **if** $v \equiv 1 \pmod{n}$ **then return** ("failure")**while** $v \not\equiv 1 \pmod{n}$ **do** $\begin{cases} v_0 \leftarrow v \\ v \leftarrow v^2 \pmod{n} \end{cases}$ **if** $v_0 \equiv -1 \pmod{n}$ **then return** ("failure") **else** $\begin{cases} x \leftarrow \gcd(v_0 + 1, n) \\ \text{return } (x) \end{cases}$ **comment:** x is a factor of n

Figure 6: Pseudocódigo del Algoritmo de las Vegas para romper RSA conocida la clave privada

2.3.1 Uso del programa: break_rsa

Se puede ejecutar el programa de dos formas: introduciendo directamente el valor de n , e y d , o bien pasándole un archivo de clave de RSA de los que generamos en el apartado opcional.

```
./break_rsa -n modulo -e exp_cifrado -d exp_descifrado
```

```
./break_rsa -f archivo_claves
```

Para ejemplos pequeños se puede utilizar el primer modo. Pero para números grandes, como el ejemplo de 4096 bits que nos pide comprobar el ejercicio, es mejor utilizar el segundo modo. La forma de operar es:

1. Se genera una clave del tamaño deseado usando el programa del apartado opcional RSA.
2. Se ejecuta este programa con el fichero de la clave para obtener los números primos.

Example 6.15 Suppose $n = 89855713$, $b = 34986517$, and $a = 82330933$, and the random value $w = 5$. We have

$$ab - 1 = 2^3 \times 360059073378795.$$

Then

$$w^r \pmod{n} = 85877701.$$

It happens that

$$85877701^2 \equiv 1 \pmod{n}.$$

Therefore the algorithm will return the value

$$x = \gcd(85877702, n) = 9103.$$

This is one factor of n ; the other is $n/9103 = 9871$. □

Figure 7: Ejemplo del Stinson de ejecución del algoritmo

En el siguiente recuadro se comprueba que el resultado de factorizar unos números pequeños en concreto coincide con el Stinson.

```
./break_rsa -n 89855713 -e 34986517 -d 82330933
---- PRIMO 1 ----
9871
---- FIN PRIMO 1 ----
---- PRIMO 2 ----
9103
---- FIN PRIMO 2 ----
```

A continuación se prueba a romper una clave de RSA con 4096 bits. La clave en cuestión es la siguiente:

```
cat key
---- MODULO ----
63814022739454014189466020614633189140425269202393388770390232420022741998594599565079842668
87022921745500370230081689726759162695797107026762674067117972854067109837395608920041170240
09660559977109865341792830226961292009062172132466052726797768260287587052664859794312635586
39475643451419234139377650556452255331206971932698178989111987023582008791281726738850707720
84616097386566914753982148186609923680596754545976436198616225367713669075196206117380091302
48346706715683631917028112205177886491732542659972337841726379057478969049567547293253179563
85836800618453091129225210947882928381198399517883656083212602260473837469185832412802703537
14770246206863622721083426240933593752091698557105588299645015454402207421964569789955527374
20486822694562216141605676883827519628460098035731824853363275640265673586575567766501076814
30056609710126095897180449735502770672369883738251711707046349408676676012815267054952690458
36562426221582691164611033125971471911303145664710952070787032542765286393717641684473134046
93466277500145117280128224680408906224334873754281976729794917730493196098898505626372894015
28583778194886223673886663856077603591240281564489887824077906590885208070536078728544256092
7391666748133854308303444417261194711
---- FIN MODULO ----
---- EXP. CIFRADO ----
58405605416862282960382121969803050448541395257317565953838974976972546759167629855986687023
39028549489410901249186328533968873517733974774743587956145123025119260035079051384206072814
01298909451050759317399204670661979623025114774003095440822003702335659925325420716296307631
12374039672960859791977221690615415052816396522691153527479077684297602111836628234733126734
08174652769860884614852229422583944590589584634086997533993887833961566020418129270271079445
5424678456555294581611448661659926111041924313617975552117684894043515922119914847866549224
03408217931850289310902262509688535376791836414852313086788946367422914616126173411523790255
37937130235609728363065963009824093187912928159813240063844329042365623750919408077064834649
94918838455438638152657995782526076515849180919505078082327313401820738947620331252424698978
90386233689816648681927587713479554916940716055676470535800660937454262365421489338290288767
9994187973418956708936269518203619884807923642697615744000615271197418421734960044647583345
25281136345250445844875015774406113935943903514489065509403585323385297480787476244305023389
79202461498416355886282947199264005921233044026685034087557636718451293444826397424298093696
9091019570145005461029179242050310121
---- FIN EXP. CIFRADO ----
---- EXP. DESCIFRADO ----
50480912032735975005905382735666885627256426389811799343510358280568299221077619405669729606
292874771358013502975989362938101117528906972786691472566293556111105117770021494207999112
99829256085181875165093945549300645299945254251797782459253054031910577999899065029735255054
01475847750862538818073056799122703797321581775985498717790039926931870908639816864364306625
78006868724647310265225875877395597424090431402303899042280828621213808916382542507193869380
52760026835961765018892775808817566034582121172613472305910418252477328768430853349867576615
224873315373853422573770457187602982884777055466125227355794021431245814421508665010537147
61078049523572606768840786856132092669059839026667546632521162742499430604985761081101445626
95298939587460006847457318267851074115281642971344796643742737792070884413236327753750647486
20799337195985755325260007139522914989484368583473245965841273113653306780744070646771009016
29527346842758646268718890326989436553811667222816401608076660127801146425505409864280381950
6625624352989842264642094716185856031806936859669697279440959220436408140690434247765423103
61757238442895286384303573451446651788735553868182392593624797902888751710730729368410193676
5564430629799024334862834792170079673
---- FIN EXP. DESCIFRADO ----
---- NUMERO DE BITS ----
4096
---- FIN NUMERO DE BITS ----
```

El contenido de estos ficheros de clave se explica en el apartado opcional de la programación de RSA.

```
./break_rsa -f key
---- PRIMO 1 ----
30719907036544169795570736281920052585319433867442944341725590220070392105255476097084692336
26842158157807691010840203498079511040506301738438680174074251108999801790393732594122033519
62748656471599452299043425120555971203790622975763174724072553323937093160987113225013382410
56782919452937764721877814230973756174333516410477929892565204803278229114051452697354211223
25640768716698817203613889011473111897745407050941089366941492996339621888419947651637223145
13205271586159634262190072334246317213341931887793313804411577152957584751027260444719577161
25972776820977530475347811089092559767703906364887894541927858149
---- FIN PRIMO 1 ----
```

```

---- PRIMO 2 ----
20772856722366162719615269621341342847311747048472241823632749562893850451438025704738619141
17075775876032491674522714334940865079692026729132455288490060534506342564218996291583466260
57477300211688598921249733606928320936599293283772096110968181096225512474196517616900856200
22676219322249886408681914907671103067076687235815956970976417361321184109323920932168821665
72950816310876256602544042883028357740545663152348114796545550303153485907986729687572710432
99754519365582869302315194705112849889638534220265148892824561956387236135097302394556390416
26175333179726810065634937623999493265356166524797059631558288139
---- FIN PRIMO 2 ----

```

2.4 Números primos

Si un observador conoce el método aleatorio que usamos ¿podría determinar fácilmente el número escogido? Para generar los números aleatorios usamos Mersenne twister con la semilla obtenida de leer en `/dev/urandom`. Si el observador conoce tanto el generador de números pseudoaleatorios como la semilla, entonces sí que puede determinar el número escogido, pues puede repetir exactamente la ejecución de nuestro programa estableciendo el generador del modo adecuado. No conocemos los pormenores del Mersenne twister, pero diría que si el observador sólo conoce el algoritmo no podría determinar el número, puesto que MT tiene buenas propiedades como generador.

¿Se verifica la conjetura de Gauss de distribución de números primos (el número de primos menores o iguales a n se comporta como $n/\log(n)$)?

Miller-Rabin es optimista en relación con los primos: clasifica como primos números que no lo son, y todos los primos los clasifica como primos. Esto nos dice que la función de distribución del número de primos menores que n estará por encima que la real, pero también nos dice que cuando el parámetro de seguridad $t \rightarrow \infty$ esta distribución se aproximará a la real por arriba. Empíricamente, hemos probado para valores de $t \in \{1, 5, 10, 20, 40\}$. Para los valores distintos de 1, las distribuciones coinciden casi exactamente. Para 1, está ligeramente por encima, confirmando lo dicho antes.

Si el test ha sido positivo en n tests consecutivos ¿qué probabilidad hay de que continúe siéndolo en los m tests siguientes? Sea $J_t \subseteq \mathbb{N} + 3$ el subconjunto de números que pasan t veces el test de Miller-Rabin. (Sabemos que es un test probabilístico, pero hagamos como si no.) Está claro que $J_{t+s} \subseteq J_t$ para todo $s \geq 1$. Además, también sabemos que el conjunto de todos los números primos $\Pi \subseteq J_t$ para cada t . Luego se cumple que

$$\lim_{t \rightarrow \infty} J_t = \bigcap_{t=1}^{\infty} J_t =: J_{\infty} \supseteq \Pi.$$

Con algo de maquinaria extra podemos ver la igualdad $J_{\infty} = \Pi$. Si tuviéramos un espacio de probabilidad (que no lo tenemos, ni podemos tener uno con sentido sobre $\mathbb{N} + 3$), la probabilidad que nos están preguntando sería

$$U(n, m) = P(J_{n+m} \mid J_n) = \frac{P(J_{n+m})}{P(J_n)}.$$

Dados los hechos que hemos expuestos hasta ahora, tenemos casi claro que se cumple

$$\lim_{n, m \rightarrow \infty} U(n, m) = \lim_{n, m \rightarrow \infty} \frac{P(J_{n+m})}{P(J_n)} = 1,$$

donde la última igualdad (y, en realidad, todo lo que se ha respondido a esta pregunta) queda por demostrar. Para seguir esta línea de ideas: probar a trabajar sobre $I_N = \{3, \dots, N\}$, para N fijo, y al final tomar límite $N \rightarrow \infty$. Con I finito sí que tenemos un espacio de probabilidad con sentido.

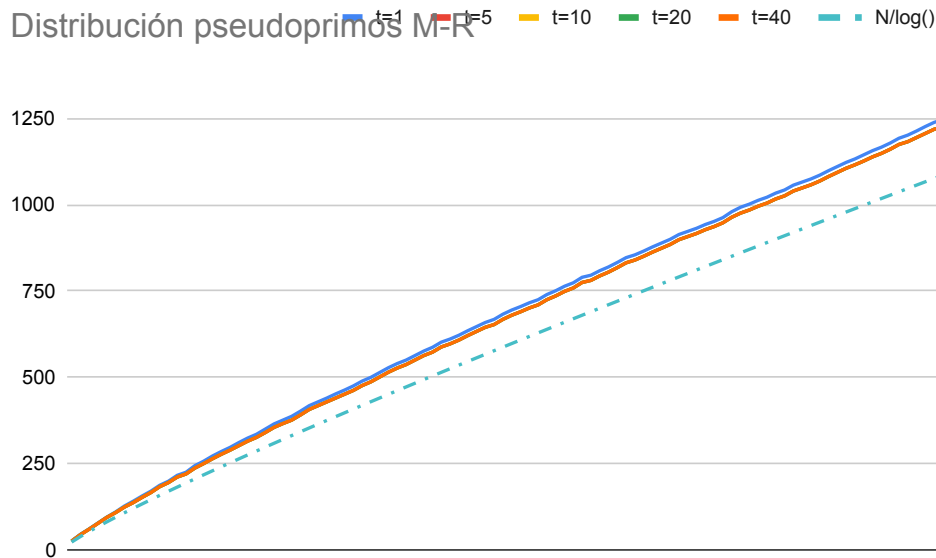


Figure 8: Gráfica de $x/\log x$ frente a la distribución de pseudoprimos que da Miller-Rabin para distintos valores del parámetro de seguridad

2.4.1 Uso del programa: test_primo

Este programa no tiene argumentos. Si es ejecutado, devuelve los datos con los que hemos construido la gráfica anterior. Calcula el número de primos hasta n , desde 5 hasta 10005 de 100 en 100, dando lugar a 100 mediciones que luego hemos graficado en Excel.

2.5 Programación del RSA

Breve resumen de RSA. Dos claves: (n, e) y (n, d) , con $n, d, e \in \mathbb{Z}$, cuyas únicas restricciones, si no nos ponemos muy rigurosos, son:

1. $n = pq$, con p y q números primos,
2. $ed \equiv 1 \pmod{\varphi(n)}$.

Si tenemos un mensaje $m \in \{0, 1, \dots, n-1\}$, el cifrado de m con la clave pública (n, e) consiste en hallar $c = m^e \pmod{n}$, y el descifrado de c con la clave privada (n, d) consiste en $u = c^d \pmod{n}$. Para ver que $m = u$ basta observar que $de = k\varphi(n) + 1$ y que $u = c^d = m^{ed} = m^{\varphi(n)+1} \equiv m \pmod{n}$. Para generar las claves seguimos el pseudocódigo de *Handbook of Applied Cryptography*, aunque el asunto no tiene mucho misterio.

8.1 Algorithm Key generation for RSA public-key encryption

SUMMARY: each entity creates an RSA public key and a corresponding private key.
Each entity A should do the following:

1. Generate two large random (and distinct) primes p and q , each roughly the same size.
2. Compute $n = pq$ and $\phi = (p-1)(q-1)$. (See Note 8.5.)
3. Select a random integer e , $1 < e < \phi$, such that $\gcd(e, \phi) = 1$.
4. Use the extended Euclidean algorithm (Algorithm 2.107) to compute the unique integer d , $1 < d < \phi$, such that $ed \equiv 1 \pmod{\phi}$.
5. A 's public key is (n, e) ; A 's private key is d .

Figure 9: Pseudocódigo para la generación de claves

Paso del fichero de texto plano a número RSA con el que cifrar. Primero un comentario sobre cómo generamos las claves: el número de bits que se indican al generar las claves es el tamaño de los **primos**, por tanto la claves es del doble de tamaño. Es decir, si ponemos el número de bits a 2048 obtendremos un módulo de 4096. Para obtener los números con los que cifra RSA, leemos del fichero de entrada bloques de tantos bits como el tamaño del módulo, o lo que es lo mismo: bloques de tantos **bytes** como el tamaño del módulo dividido entre 8. Usamos `mpz_import` y `mpz_export` del modo adecuado para tratar con los enteros de `gmp`. El modo de operación es ECB pues no hacemos encadenamiento.

2.5.1 Uso del programa: RSA

Hemos simplificado un poco el formato del programa que se propone en el enunciado. Dado que casi siempre ejecutamos estos programas para cifrar archivos, se han puesto como obligatorios los ficheros de entrada y de salida, de modo que no se puede usar la entrada ni la salida estándar para el uso de este programa. Hay tres modos de operación: generar las claves, cifrar y descifrar.

```
./RSA -K -n numero_bits -s sec [-o fichero_salida]

./RSA -E {-n modulo -e e_cifrado | -f fichero_claves} {-i fichero_entrada} {-o fichero_salida}

./RSA -D {-n modulo -d e_descifrado | -f fichero_claves} {-i fichero_entrada} {-o fichero_salida}
```

Los parámetros significan lo mismo que en el enunciado, y el parámetro de seguridad de la clave es el mismo que en el programa para generar primos, a saber, un número positivo s que se relaciona con la probabilidad deseada de fallo haciendo 10^{-s} .

A continuación se muestra un ejemplo de fichero de clave que genera el programa. Estos ficheros contiene en base 10 el módulo, los dos exponentes y el tamaño de la clave. Nótese como se ha llamado al programa con $n := 1024$ y en el fichero de clave aparece un tamaño de 2048. Esto se debe a que 1024 es el tamaño de los primos generados, y no de la clave. Estos ficheros se pueden usar también en el programa para factorizar el módulo de RSA conociendo la clave privada.

```
./RSA -K -n 1024 -t 20 -o key2048
cat key2048
---- MODULO ----
28798158352949180297961859478749404724115977924032045185624064092243863762959107649786052558
34055043445588645346387536938141926297567300266878971993698599756661716159430614064903896032
43489164937660451820226586485193488852166114567999813908563039710374274535672565769917264127
03344326941907359994073922005988938369867120312607678546874588298491473249792050235634181385
88298239899791973725450701611457337763713523060260708179902091558023297693325585001995925867
19785699271020551366894912774145041495236336022173120569369522154574550673958621884636481222
94389208275972650225761861379590696210110245731441859804939743381
---- FIN MODULO ----
---- EXP. CIFRADO ----
11830619276203689688736075628019649670454555905608682281021794221551198649821371018201232287
22373988590901022068063796988718098198589165131668386114111264000194308309871637939414117910
76765244027265566123374688217456757104705750741502784943091631044399952298217639013477816293
23312980829293772318770575341073434685413657061075339248270531360399417977327123329027601306
72192426748046903770278436892091358138176764260453122710146786631624098821827656343660755378
62563229775412908563649893329965619107758873889427235487906711841424365912478521025661526415
42948831393024616382646700697581325889091054825855160235018284349
---- FIN EXP. CIFRADO ----
---- EXP. DESCIFRADO ----
50420822105605848452313892811987408042265680504280391122316434708578038244890776040671544743
34601557426497657548925448587932784064799956905963796067974361468908189276976614083698273833
25957338958834544819241739671424486744955075632280093781629400429784287600520324140142981296
5455302900275056011370306213959871486660177920673888340332380045271883818517529660317043279
08544597889253445010214253283717133672265850024771754338203071414019466226396677694989309339
21526400827059224680121612680478955065902983042843008157939566069202944838671872984815261551
3732937384841780701106444399803896992936985330729610293533836305
---- FIN EXP. DESCIFRADO ----
---- NUMERO DE BITS ----
2048
---- FIN NUMERO DE BITS ----
```

Ahora mostramos el resultado del programa para cifrar y descifrar la Sonatina de Ruben Darío.

```
./RSA -C -f key2048 -i sntn -o sntncif
./RSA -D -f key2048 -i sntncif -o sntncifdes
head -n 2 sntncifdes
La princesa esta triste... Que tendra la princesa?
Los suspiros se escapan de su boca de fresa,
tail -n 3 sntncifdes
```

y que llega de lejos, vencedor de la Muerte,
a encenderte los labios con un beso de amor.